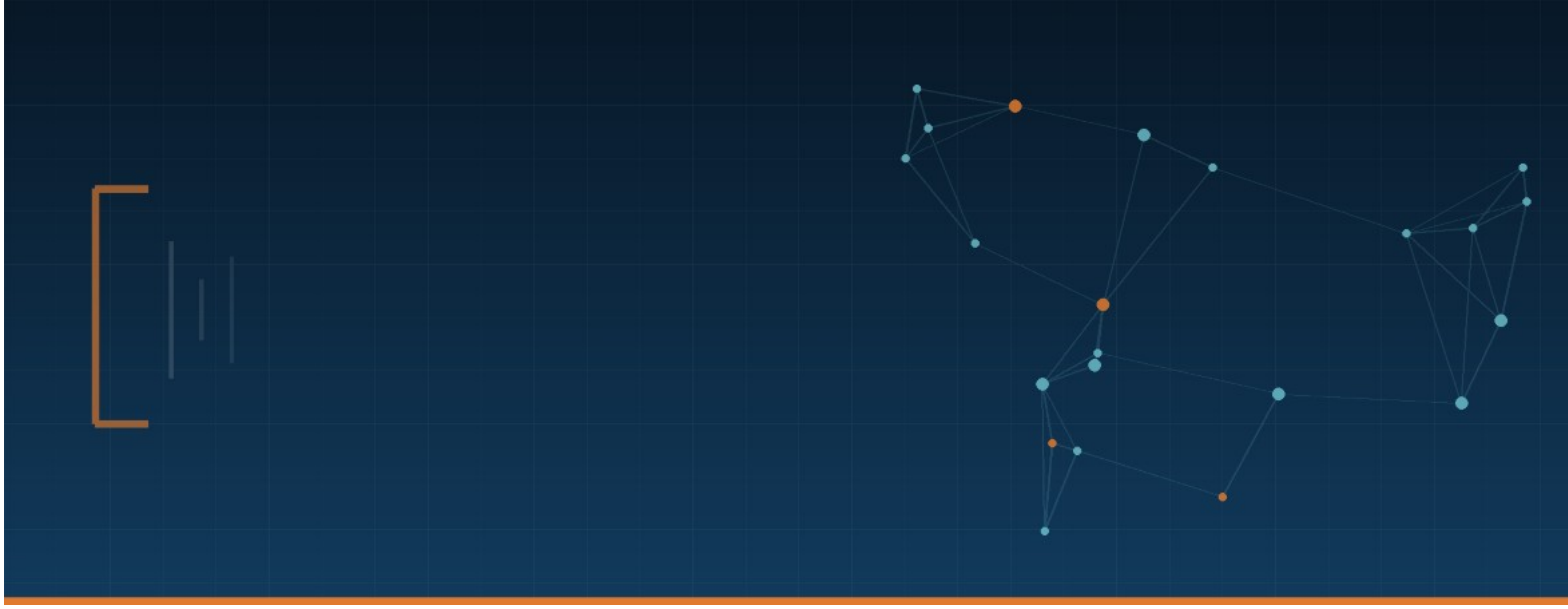


RESEARCH DATA HANDBOOK • 2026

网络数据获取与爬虫入门指南

面向经济学研究的零基础教学版

从官方 API 到 requests + BeautifulSoup、Scrapy、Playwright，按“数据在哪里、规模有多大”选择最合适的取数方案。全书抓取案例统一以豆瓣电影 Top 250 为例，先理解网页结构，再写第一行代码。

本指南适合谁

经济学研究者、研究生与研究助理；会使用浏览器和 Excel 即可，不需要编程基础。所有示例都围绕“可复现的研究数据”展开，而不是追求抓取量。

适用对象	经济学研究者、研究生、研究助理；零编程基础
前置知识	会安装软件、会用浏览器开发者工具（第 3 章会教）
配套内容	豆瓣 Top250 贯穿案例 • 3 张流程图 • 5 份检查清单
版本	v1.0 • 2026 年 10 月 • 组会内部资料

组会资料 • 经济学研究数据获取系列

目录

学习地图

这本指南按“先判断、再动手”的顺序组织。点击章节标题或页码可直接跳转；PDF 左侧书签还可展开到二级标题。

章	标题	内容	页
前言	为什么写这本指南	目标、学习路径、环境与约定	1
1	数据获取的整体思路	五层阶梯、请求循环、项目生命周期	4
2	官方 API：最优先的方案	URL、JSON、分页、限流、字段与 vintage	8
3	HTML 网页结构简介	豆瓣 Top250 的标签、DOM 树与选择器	14
4	requests + BeautifulSoup	抓取豆瓣 Top250、翻页、CSV/Parquet	22
5	Scrapy	豆瓣 Top250 Spider、Pipeline、增量抓取	32
6	Playwright	豆瓣 Top250 + 动态页对照实例	39
7	组合方案与工程化	豆瓣 Top250 项目骨架、配置、调度	46
8	数据质量与可复现性	榜单快照、哈希、校验、复现包	51
9	合规、伦理与风险	robots、条款、版权、个人信息	55
10	常见问题与排查手册	403/429、空列表、乱码、超时	59
11	练习与进阶路线	5 个豆瓣 Top250 练习、30 天计划	62
A	环境安装与命令速查	venv、pip、浏览器安装	65
B	选择器与代码片段速查	豆瓣 Top250 的四种写法	67
C	术语表	常用名词	69
D	经济学数据源入口	API、批量、统计网站	70
E	检查清单	抓取前、中、后	71

目录页码为正文页码，与页脚一致。若在 PDF 阅读器中点击无反应，请使用左侧书签导航。

前言

为什么写这本指南

经济学研究越来越依赖“自己取数”：宏观数据要更新到最新月份，政策文本要覆盖十年，企业公告要按行业拼接，地方统计年鉴常常只有网页表格而没有 API。于是，研究者不得不面对 HTML、JSON、分页、编码和反爬。但大多数教程从“如何绕过封锁”讲起，忽略了更重要的问题：这个数据源是否本来就有更好的获取方式？

> **读完这本指南，你应该能够**
>
> - 判断一个数据源该用 API、静态抓取、Scrapy 还是 Playwright；
> - 读懂 HTML 的树状结构，并用 CSS 选择器稳定地定位数据；
> - 独立完成“请求—解析—校验—保存”的最小闭环；
> - 为数据留下原始快照、抓取时间和校验记录，保证研究可复现。

0.1 这本指南解决什么问题

它不追求“抓遍全网”，而是围绕经济学研究中最常见的四类数据：宏观指标、统计表格、公告与报告、文本与新闻。对每一类数据，先给出优先级最高的获取方式，再给出在没有 API 时的实现路径。

书中所有代码都以“能跑、能改、能复现”为标准。第 3—6 章的抓取案例统一使用豆瓣电影 Top 250（`https://movie.douban.com/top250`）；API 示例使用世界银行和 FRED 的公开接口。你可以把 URL 换成自己的数据源，但请先完成第 9 章的合规检查。

- **不教**：绕过登录、付费墙、验证码、平台风控，或大规模抓取个人隐私数据。
- **重点教**：如何判断数据在哪里、如何写出稳定的选择器、如何把一次性脚本变成可维护的项目。

0.2 谁适合读，怎么读

目标读者是经济学研究者、研究生和研究助理。你不需要编程基础，但需要会安装软件、会用浏览器。如果你已经会 Python，可以跳过第 4 章的基础语法，直接看选择器、翻页和存储部分。

表 0-2 推荐学习路径

阶段	读什么	完成后你能做什么
① 基础	第 1—3 章	判断数据源类型；读懂页面结构；写出第一个选择器
② 单页	第 4 章	抓一个静态表格，清洗后保存为 CSV/Parquet
③ 规模	第 5 章	用 Scrapy 跑多页任务，断点续爬，输出 JSONL
④ 动态	第 6 章	用 Playwright 等待渲染、点击、下载、拦截 XHR
⑤ 工程	第 7—9 章	组织项目、保存快照、做校验、过合规
⑥ 巩固	第 10—11 章	独立排查常见错误，完成 5 个练习

0.3 动手前的环境

书中的命令默认在 Windows PowerShell 中执行；macOS/Linux 只需把虚拟环境激活命令换成 `source .venv/bin/activate`。完整安装步骤见附录 A。

表 0-3 最小工具清单

工具	建议版本	用途
Python	3.10 及以上	运行所有示例
VS Code	最新稳定版	编辑代码、选择解释器、打开终端
Chrome / Edge	最新稳定版	开发者工具（Elements、Network）
pip + venv	Python 自带	隔离项目依赖
Git（可选）	最新稳定版	版本管理；不是爬虫必需

0.4 本书约定

- 代码块中的 `YOUR_KEY`、`https://example.com` 是占位符，运行前要替换。
- 示例优先使用 `jsonl`、`csv`、`parquet` 三种格式；原始响应单独存放，不覆盖。
- “静态页面”指第一次 HTTP 响应里就有数据；“动态页面”指数据由 JavaScript 再次请求后渲染。
- 库的版本会变化。遇到报错，先看官方文档的当前参数，再对照本书的排查表。
- 所有抓取都应设置超时、限速和重试；不要把“能跑通”当成“可以长期运行”。

0.5 合规声明

先合规，再取数

- 本指南只讨论公开、允许访问的数据，不提供绕过登录、付费墙、验证码或平台风控的方法。
- 抓取前核对 robots.txt、网站服务条款、版权与数据库权利，以及个人信息保护要求。
- 能通过官方 API、批量下载、机构订阅或数据购买获得的，优先走授权渠道。
- 控制请求频率，标明身份与联系方式；被拒绝访问时停止，而不是换代理继续。

这本书把“合规”放在技术之前，不是为了增加负担，而是因为研究数据必须能经得起复现和审查。一个能跑但来源不清、条款不明的数据集，很难成为可信研究的基础。

关于贯穿案例：豆瓣电影 Top 250

- 豆瓣 robots.txt 未禁止 `/top250`；本书只取榜单公开字段（排名、片名、评分、短评、链接），不涉及登录和个人信息。
- 示例建议每次请求间隔 5 秒、只抓 10 页；若返回 403、验证码或要求登录，立即停止，不要换代理硬绕。
- 该案例用于练习 HTML 解析与翻页，不代表豆瓣授权；正式研究请优先使用官方 API、授权数据或商业数据库。

三条阅读建议

- 先读第 1—3 章，再选工具；不要从 Scrapy 或 Playwright 开始。
- 每学一种方法，就在自己的数据源上试 10 条；不要只抄代码。
- 把原始响应、脚本和校验结果放在同一个项目目录，两周后还能复现。

第 1 章

数据获取的整体思路

先判断数据在哪里，再决定用什么工具；大多数“爬虫难题”，其实是选错了入口。

本章目标

- 用三个问题判断数据源类型
- 理解请求—响应—解析—保存的循环
- 为一个研究项目安排取数、校验和存档的顺序

经济学研究中的数据获取，通常不是“把一个网页抓下来”这么简单。你面对的可能是一个 API、一张嵌在网页里的表格、一份需要登录才能下载的 Excel，或者一个由 JavaScript 动态渲染的看板。不同来源对应不同的工具，选错工具会让后续维护成本成倍增加。

1.1 先问三个问题

在安装任何库之前，先对数据源做一次“体检”。下面三个问题，基本决定了技术路线。

1. **有没有官方 API、批量下载或机构订阅？** 如果有，优先用。它通常字段稳定、有文档、可引用。
2. **数据是否直接出现在第一次响应的 HTML 里？** 如果是，`requests + BeautifulSoup` 就够了。
3. **规模有多大、页面有多少、是否要周期性更新？** 单站少量页面用脚本；多页、多站点、周期任务用 Scrapy；只有确实依赖 JavaScript 渲染时，才上 Playwright。

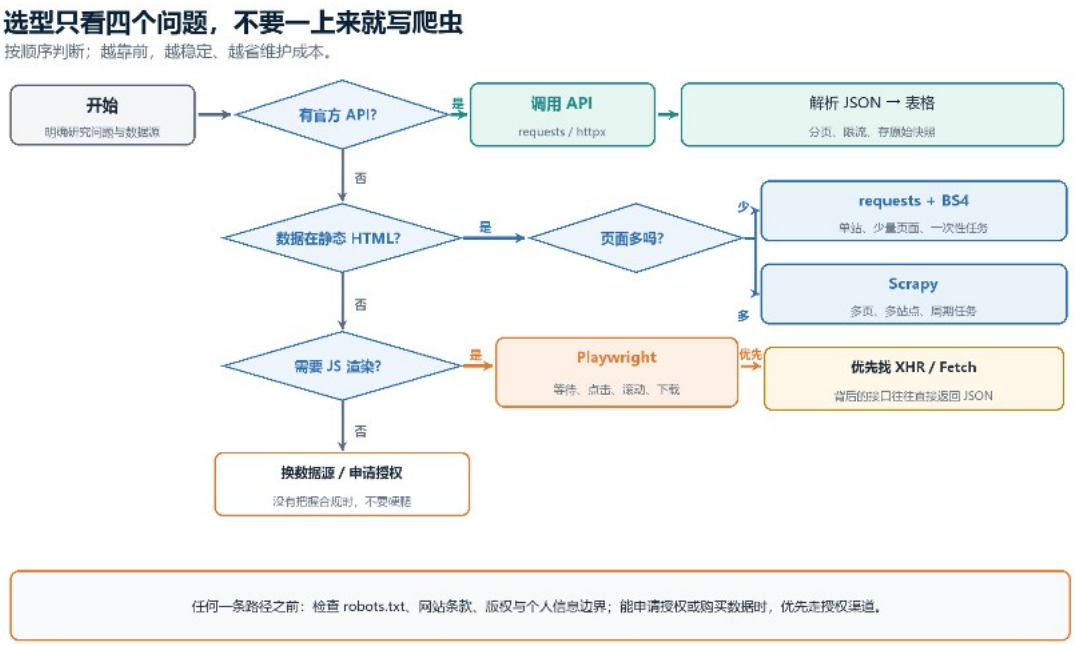


图 1-1 数据获取方案的判断顺序

1.2 五层阶梯：从最稳定到最脆弱

把可选方案按稳定性排序，你会发现“爬虫”只是其中一层。越靠上的方案，越不依赖网页外观，也越容易写进论文的数据附录。

表 1-1 数据获取方案的五层阶梯

层级	方案	稳定性	维护成本	典型场景
1	官方 API / 开放数据	高	低	世界银行、FRED、IMF、OECD
2	商业数据库 / 数据终端	高	中（费用）	Wind、CEIC、CSMAR、Datastream
3	批量文件 / 年鉴 / 调查数据	高	低	统计年鉴、Eurostat bulk、CFPS
4	静态网页抓取	中	中	地方统计局表格、公告列表
5	浏览器自动化 / 反爬对抗	低	高	JS 渲染、登录后下载、复杂交互

研究提示

- 同一份数据往往有多个入口。先花 30 分钟找 API、数据附录或批量下载，可能省掉后面 3 天的爬虫维护。
- 如果数据来自商业数据库，先确认订阅许可是否允许你导出、保存和二次分析；不要用爬虫绕过许可。

1.3 爬虫到底在做什么

无论用哪个库，底层都是同一个循环：你的程序构造一个 HTTP 请求，服务器返回响应，程序从 HTML 或 JSON 中取出字段，校验后保存，再对下一条记录重复。所谓“爬虫框架”，只是把这个循环中的并发、重试、去重、排队和存储做得更可靠。

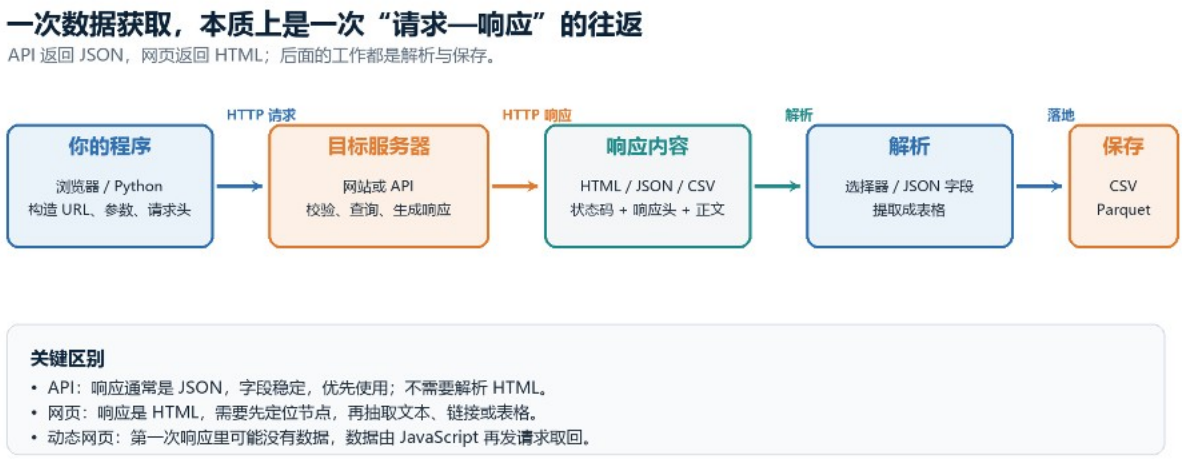


图 1-2 一次请求—响应—解析—保存的往返

API 和网页的差别，主要在“响应内容”这一步：API 直接给你 JSON，字段清楚；网页给你 HTML，需要先定位节点。动态网页则更绕一层：第一次响应里没有数据，浏览器执行 JavaScript 后再发一次请求，数据才出现。第 3 章会专门讲如何判断这三种情况。

1.4 一个研究项目的取数流程

把取数当成一个小型数据工程，而不是一次性脚本。下面的顺序适用于从几十行到几十万行的项目，只是每一步的复杂度不同。

表 1-2 取数项目的八个阶段

阶段	要做什么	产出/证据
① 定义	明确变量、单位、频率、时间范围、覆盖范围	数据需求表
② 登记	记录来源、URL、许可、更新频率、联系人	数据源登记表
③ 试抓	先抓 10—50 条，确认字段、编码、翻页和空值	样本数据 + 截图
④ 解析	写选择器/字段映射，处理缺失与类型	解析脚本 + 单元测试
⑤ 校验	行数、唯一键、范围、时间连续性、交叉来源	校验报告

阶段	要做什么	产出/证据
⑥ 全量	限速、重试、断点续跑、日志	原始快照 + 清洗数据
⑦ 存档	快照、哈希、脚本版本、README	复现包
⑧ 引用	数据来源、版本、访问日期、许可	论文数据附录

1.5 研究者最常见的五个误区

- **一上来就学框架。** 如果连 HTML 的父子节点都看不清，Scrapy 的 `css()` 只会让你更困惑。先学第 3 章。
- **把网页当 API。** 页面结构一改，脚本就报废；能找到背后的 JSON 接口，就不要解析渲染后的 HTML。
- **不保存原始响应。** 只存清洗后的 CSV，等于放弃了复现和排查的证据。原始 HTML/JSON 一定要留。
- **用固定睡眠代替等待。** `time.sleep` 既慢又不稳；Playwright 应该等待具体元素或响应，而不是猜时间。
- **抓完不校验。** 行数对不上、单位混用、日期错位，往往比“抓不到”更危险。第 8 章给出校验清单。

1.6 小结

这一章只做一件事：把“写爬虫”拆成“判断数据在哪里”和“选择合适的获取方式”。下一章从最优先的方案——官方 API——开始；即使你最终要做网页抓取，理解 API 的请求/响应/字段思维，也会让后面的解析更容易。

> **关于本书的贯穿案例**
>
> - 第 2 章的 API 示例保留世界银行和 FRED，因为豆瓣没有公开 API；这正好说明“有 API 优先用 API”。
> - 从第 3 章起，所有抓取实现统一以豆瓣电影 Top 250 为例：同一份数据，分别用 HTML、requests、Scrapy、Playwright 完成。

第 2 章

官方 API：最优先的方案

API 是数据提供方写好的“菜单”：你按规则点菜，它按字段返回。稳定、可引用、维护成本最低。

本章目标

- 理解 endpoint、参数、状态码和 JSON
- 用 requests 调通一个无 key 和有 key 的 API
- 处理分页、限流、缺失值与数据版本

只要数据源提供 API，就应该优先使用。原因很实际：字段有文档、结构稳定、通常允许批量获取，而且你不需要猜测网页的 CSS 类名。对经济学研究来说，API 还能提供元数据（单位、季节调整、修订时间），这些信息直接影响变量能否比较。

2.1 API 是什么：一次点菜的过程

把 API 想成餐厅：菜单告诉你有哪些菜（endpoint）、每道菜怎么点（参数）、价格和分量（字段与单位），服务员把请求传给厨房（服务器），再把菜端回来（响应）。你不需要进厨房，也不需要知道厨房怎么运作。

表 2-1 API 的六个基本概念

概念	含义	例子
Endpoint	一个具体的接口地址	`/v2/country/CHN/indicator/NY.GDP.MKTP.CD`
Method	请求动作，读取数据通常用 GET	`GET`
Query 参数	放在 `?` 后面的键值对	`format=json&per_page=100`
Header	请求头，常放 key、UA、Accept	`Authorization: Bearer ...`
Status code	服务器对这次请求的结果码	`200`、`401`、`429`
Response	返回的正文，常见为 JSON	`{"observations": [...]}`

一句话区分 API 与网页

- API 返回“数据 + 字段名”，网页返回“数据 + 排版”。前者的契约是字段，后者的契约是页面结构。

2.2 解剖一个 API 请求

下面是一个典型的 API URL。把它拆开看，你会发现它和浏览器地址栏没有本质区别。

代码 2-1 一个带查询参数的 API URL

```
https://api.stlouisfed.org/fred/series/observations
  ?series_id=GDPC1
  &api_key=YOUR_KEY
  &file_type=json
  &observation_start=2000-01-01
```

表 2-2 URL 的六个部分

部分	内容	作用
协议 + 主机	`https://api.stlouisfed.org`	去哪里取
路径	`/fred/series/observations`	取哪类资源
查询参数	`series_id=GDPC1`	取哪一条序列
认证	`api_key=YOUR_KEY`	证明你有权限
格式	`file_type=json`	要求返回 JSON
时间范围	`observation_start=2000-01-01`	限制观测区间

HTTP 状态码是排查问题的第一线索。`200` 表示成功；`400` 多半是参数写错；`401/403` 是 key 或权限问题；`404` 是路径或资源不存在；`429` 是请求太快；`5xx` 是服务器端问题，适合重试。

2.3 第一个 API：世界银行（不需要 key）

世界银行的数据 API 允许直接查询国别指标，适合练习。下面取中国 2000 年以来的 GDP（现价美元）。注意它的响应是一个列表：第一个元素是元数据，第二个元素才是观测值。

代码 2-2 调用世界银行 API 并转成 DataFrame

```python
import requests
import pandas as pd

url = "https://api.worldbank.org/v2/country/CHN/indicator/NY.GDP.MKTP.CD"
params = {"format": "json", "per_page": 100, "page": 1}

resp = requests.get(url, params=params, timeout=30)
resp.raise_for_status()
payload = resp.json()

meta, rows = payload[0], payload[1]
df = pd.DataFrame(rows)[["date", "value"]]
df["value"] = pd.to_numeric(df["value"], errors="coerce")
df["date"] = pd.to_datetime(df["date"], format="%Y")
print(df.sort_values("date").tail())
```

如果 `payload[1]` 是 `None`，通常是国家代码、指标代码或分页参数有问题。先打印 `payload` 看错误信息，不要直接对 `None` 取下标。

代码 2-3 先检查响应结构，再解析

```
if not isinstance(payload, list) or len(payload) < 2 or payload[1] is None:
    raise ValueError(f"API 返回异常: {payload}")
```

2.4 带 key 的 API：以 FRED 为例

有些 API 需要注册并携带 key。key 是凭证，不是可以写进论文或 GitHub 的配置。正确做法是放在环境变量里，代码只读变量名。

代码 2-4 在终端设置环境变量

```
# Windows PowerShell
$env:FRED_API_KEY = "你的 key"

# macOS / Linux
export FRED_API_KEY="你的 key"
```

代码 2-5 用环境变量调用 FRED API

```
import os
import requests
import pandas as pd

api_key = os.environ.get("FRED_API_KEY")
if not api_key:
    raise SystemExit("请先设置环境变量 FRED_API_KEY")

url = "https://api.stlouisfed.org/fred/series/observations"
params = {
    "series_id": "GDPC1",
    "api_key": api_key,
    "file_type": "json",
    "observation_start": "2000-01-01",
}
resp = requests.get(url, params=params, timeout=30)
resp.raise_for_status()

obs = resp.json()["observations"]
df = pd.DataFrame(obs)[["date", "value"]]
df["value"] = pd.to_numeric(df["value"], errors="coerce")
df["date"] = pd.to_datetime(df["date"])
print(df.head())
```

不要把 key 写进代码

- 不要提交 `.env`、`config.py` 里的明文 key，也不要把 key 贴在聊天记录或论文附录里。
- 如果 key 泄露，立刻在服务商后台撤销并重新生成；不要试图“只在本地用”就忽略风险。

2.5 分页、限流与重试

大多数 API 不会一次返回全部数据。分页参数可能是 `page/per_page`、`offset/limit`、`cursor` 或 `next` 链接。先读文档，确认分页方式，再写循环。下面的例子用世界银行的 `page/per_page`。

代码 2-6 带限速与分页的 API 循环

```python
import time
import requests

def fetch_world_bank(indicator, country="CHN", max_pages=50):
    url = f"https://api.worldbank.org/v2/country/{country}/indicator/{indicator}"
    params = {"format": "json", "per_page": 100, "page": 1}
    rows = []
    for _ in range(max_pages):
        resp = requests.get(url, params=params, timeout=30)
        if resp.status_code == 429:
            time.sleep(int(resp.headers.get("Retry-After", "30")))
            continue
        resp.raise_for_status()
        meta, data = resp.json()
        if not data:
            break
        rows.extend(data)
        if params["page"] >= meta["pages"]:
            break
        params["page"] += 1
        time.sleep(0.3)
    return rows
```

对于偶发的 5xx 或网络抖动，可以在 `requests.Session` 上挂载重试策略。重试要有上限，并且只对幂等的 GET 请求使用；不要对“创建订单”这类写操作盲目重试。

代码 2-7 给 Session 配置有限重试

```python
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retry = Retry(
    total=3,
    backoff_factor=1.0,
    status_forcelist=[429, 500, 502, 503, 504],
    allowed_methods=["GET"],
)
session.mount("https://", HTTPAdapter(max_retries=retry))
```

限流不是“礼貌装饰”，而是避免被封禁、保证数据完整性的工程手段。把请求间隔、并发数和每日上限写进配置，不要靠感觉。遇到 `429` 先读 `Retry-After`，再退避，而不是立刻换 IP。

2.6 从 JSON 到表格

API 返回的 JSON 可能是嵌套的。`pandas.json_normalize` 可以把嵌套字段摊平成列，但在摊平之前，最好先把原始 JSON 原样保存，便于以后重新解析。

代码 2-8 保存原始 JSON，再摊平成分析表

```
import json
import pandas as pd

with open("data/raw/wb_gdp.json", "w", encoding="utf-8") as f:
    json.dump(payload, f, ensure_ascii=False, indent=2)

df = pd.json_normalize(rows)
keep = ["date", "value", "indicator.id", "indicator.value", "country.id"]
df = df[keep].rename(columns={"indicator.id": "indicator", "country.id": "country"})
df["value"] = pd.to_numeric(df["value"], errors="coerce")
df["date"] = pd.to_datetime(df["date"], format="%Y")
df = df.sort_values(["country", "date"])
df.to_parquet("data/interim/wb_gdp.parquet", index=False)
```

注意 `value` 可能是字符串、`null`、`".."` 或空字符串。统一用 `pd.to_numeric(..., errors="coerce")` 转成数值，把无法解析的标记为缺失，并记录缺失比例。不要用 `fillna(0)` 掩盖缺失。

2.7 API 不止一种形态

表 2-3 常见的 API 形态

形态	特征	常见来源	读法
REST / JSON	一个 URL 一组参数，返回 JSON	世界银行、FRED	最直接，优先掌握
SDMX	统计机构的标准，支持 XML/JSON	IMF、OECD、BIS、Eurostat	先查 dataflow 和 key 结构
Bulk 下载	整包 ZIP/CSV，按年份或主题	Eurostat、OECD、各国统计局	适合全量，注意文件说明
GraphQL	单端点，按查询语句取字段	部分商业数据服务	先学 schema，再写 query
OData	带过滤、分页、元数据的协议	部分国际组织	注意 `$filter`、`$top`

如果只是探索性取数，DBnomics 这类聚合器很方便；但正式研究应尽量回到原始发布机构，核对指标定义、修订时间和许可。聚合器可以帮你发现数据，不一定能作为最终引用来源。

2.8 研究级细节：单位、版本与缺失

API 让取数变容易，但也让“拿错变量”变得更隐蔽。下面四项必须在数据字典里写清楚。

- **单位与价格基础**：现价/不变价、本币/美元、年化/季度、指数基期。
- **季节调整**：原始值、季调值、同比/环比，不能混在同一条序列里。
- **数据版本（vintage）**：宏观数据会修订。需要“当时能看到的数据”时，用 ALFRED 或实时数据库，并把 `realtime_start`、`realtime_end` 记录下来。
- **缺失编码**：`null`、`".."`、`"NA"`、空字符串的含义可能不同；保留原始标记，再统一转缺失。

写进数据附录

- 数据源名称、API endpoint、指标代码、检索日期、数据版本/修订日期、单位、许可。
- 如果使用了实时数据，注明 vintage 日期；否则读者无法判断你是否用了“事后修订”的数据。

2.9 API 检查清单

表 2-4 API 取数检查清单

检查项	通过标准
文档	读过节流、分页、字段、错误码和许可说明
认证	key 放在环境变量；代码库中没有明文凭证
分页	确认总页数/游标；不会漏掉最后一页
限流	有间隔、并发上限和 429 退避
原始数据	每次响应保存 JSON + 抓取时间 + URL
字段	单位、频率、季节调整、缺失编码已记录
版本	需要时保存 vintage 日期；知道数据会修订
引用	有 DOI/URL、发布机构、访问日期

2.10 API 不够用时

现实中的经济学数据，常常没有 API：地方统计年鉴是网页表格，银行年报是 PDF，政策文本是新闻页，企业公告是带附件的详情页。这时才进入网页抓取。

但在写第一行爬虫之前，必须补上一门基础课：HTML 网页结构。否则你只能复制别人给的选择器，页面一改就束手无策。下一章从“浏览器如何把 HTML 变成页面”开始，用豆瓣 Top250 讲清“如何用 CSS 选择器定位一部电影”。

第 3 章

HTML 网页结构简介

爬虫的“眼睛”不是正则表达式，而是 DOM 树。看懂标签的嵌套关系，选择器就有了方向。

本章目标

- 说清浏览器、HTML、DOM、CSS 之间的关系
- 用豆瓣 Top250 认识榜单页的标签结构
- 用开发者工具找到稳定选择器，并判断页面是静态还是动态

这一章是后面所有抓取方法的公共基础。你不需要成为前端工程师，但必须能回答三个问题：数据在哪个标签里？这个标签有什么可识别的属性？换一个页面，这个选择器还成立吗？从本章开始，所有抓取示例都围绕同一个目标：豆瓣电影 Top 250。

3.1 从输入网址到看见页面

在地址栏输入 URL 后，浏览器先做 DNS 解析，找到服务器；然后建立 TCP/TLS 连接，发送 HTTP 请求；服务器返回 HTML；浏览器解析 HTML，构建 DOM 树，加载 CSS 和 JavaScript，最后把页面画出来。Python 的 `requests` 通常只完成“请求 + 拿到 HTML”这一步，不执行 JavaScript，也不渲染页面。

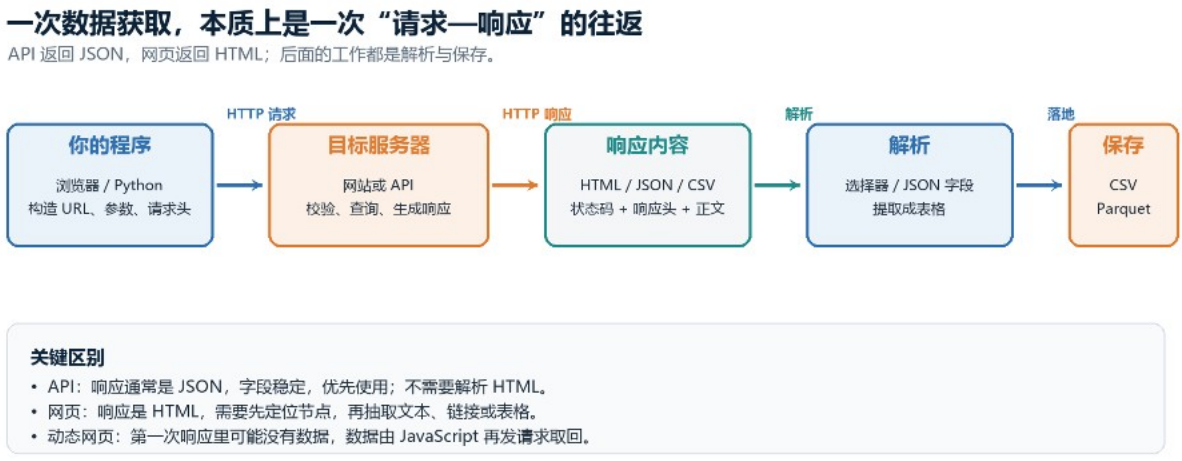


图 3-1 浏览器取回 HTML，再解析成 DOM 并渲染

最重要的一句话

- `requests` 拿到的是服务器第一次返回的 HTML；浏览器 Elements 面板看到的是 JavaScript 执行后的 DOM。
- 豆瓣 Top250 是服务端渲染的静态榜单：第一次 HTML 里就有片名和评分，所以 `requests` 就能拿到。

3.2 一个最小的 HTML 文档：豆瓣榜单条目

下面是从豆瓣 Top250 页面简化出来的一个榜单条目。它保留了真实页面最关键的结构：一个有序列表、一个电影卡片、排名、标题、评分、短评和链接。

代码 3-1 豆瓣 Top250 榜单条目的最小 HTML

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>豆瓣电影 Top 250</title>
</head>
<body>
  <ol class="grid_view">
    <li>
      <div class="item">
        <div class="pic">
          <em class="">1</em>
          <a href="https://movie.douban.com/subject/1292052/">
            <img alt="肖申克的救赎" src="poster.jpg">
          </a>
        </div>
        <div class="info">
          <div class="hd">
            <a href="https://movie.douban.com/subject/1292052/">
              <span class="title">肖申克的救赎</span>
              <span class="title"> / The Shawshank Redemption</span>
            </a>
          </div>
          <div class="bd">
            <p>导演: 弗兰克·德拉邦特<br>1994 / 美国 / 犯罪 剧情</p>
            <div class="star">
              <span class="rating_num" property="v:average">9.7</span>
              <span>3353799 人评价</span>
            </div>
            <p class="quote"><span class="inq">希望让人自由。</span></p>
          </div>
        </div>
      </div>
    </li>
  </ol>
</body>
</html>
```

表 3-1 豆瓣榜单条目的结构

部分	作用	对爬虫的意义
`<ol class="grid_view">`	有序列表，装 25 部电影	榜单的总容器
`<li>`	一部电影	逐条循环的单位
`div.pic > em`	排名	第 1 部、第 2 部……
`div.hd > a`	详情页链接	取 `href`，可进入电影详情
`span.title`	片名（中文/原名）	第一个通常是最常用的中文名
`span.rating_num`	评分	9.7、9.6……
`p.quote > span.inq`	短评	一句话影评，可能缺失
`div.paginator`	分页	下一页链接，10 页共 250 条

3.3 标签、属性与文本

HTML 用“标签”描述内容的结构。大多数标签成对出现：`<h1>` 开始，`</h1>` 结束；中间是文本或子标签。`<meta>`、`<img>`、`<br>`、`<input>` 这类是空元素，不需要闭合。

标签上的“属性”提供额外信息。对豆瓣榜单最重要的三个属性是：`class`（用于样式和定位）、`href`（详情页链接）和 `property`（评分字段的语义标记）。

代码 3-2 豆瓣榜单里最常见的属性写法

```
<ol class="grid_view">...</ol>
<li><div class="item">...</div></li>
<span class="rating_num" property="v:average">9.7</span>
<a href="https://movie.douban.com/subject/1292052/">...</a>
```

表 3-2 豆瓣榜单里的常见标签

标签	含义	在豆瓣榜单里装什么
`<ol>`/`<li>`	有序列表/列表项	25 部电影，整体有序
`<div>`	通用容器	卡片、图片区、信息区
`<a>`	链接	电影详情页、下一页
`<span>`	行内容器	片名、评分、短评
`<em>`	强调/编号	排名数字
`<p>`	段落	导演、年份、地区、类型
`<img>`	图片	海报地址（不是本节重点）
`<script>`	脚本	页面统计与广告；榜单数据不在这里

不要被 class 名字骗了

- `class="item"`、`class="title"` 这类名字直接对应业务含义，比较稳定。
- `class="pic"`、`class="bd"` 更像布局缩写；它们能用，但页面改版时不如语义类名稳。

3.4 DOM 树：把榜单看成父子关系

浏览器把 HTML 解析成一棵树：`html` 是根，`body` 下面有 `ol.grid_view`；`ol` 下面有 25 个 `li`；每个 `li` 下面有 `div.item`；`item` 又分成 `pic` 和 `info`；`info` 里有 `hd` 和 `bd`；`bd` 里有评分 `star` 和短评 `quote`。所谓“选择器”，就是从根走到目标节点的一条路径。



图 3-2 HTML 的 DOM 树与选择器路径

节点之间有四种关系：父节点、子节点、兄弟节点、后代节点。CSS 选择器里的空格表示“后代”，`>` 表示“直接子节点”，`+` 表示“紧邻的下一个兄弟”，`~` 表示“后面的所有兄弟”。

代码 3-3 豆瓣榜单的 CSS 选择器写法

```
ol.grid_view > li             /* 直接子节点：25 个电影条目 */
ol.grid_view > li .info .hd   /* 后代：每个条目的标题区 */
.quote .inq                   /* 短评文本 */
```

浏览器会自动补全一些标签，也会把不规范的 HTML 规整成树。写选择器时以 Elements 面板为准，但保存原始 HTML 时以服务器响应为准。

3.5 在豆瓣页面上，数据分别在哪里

表 3-3 豆瓣 Top250 的七个字段位置

数据	页面位置	选择器
排名	`div.pic > em`	`.pic em`
片名	`div.hd a span.title`	`.hd a .title`
评分	`span.rating_num`	`.rating_num`
评价人数	`div.star` 的最后一个 `span`	`.star span:last-child`
短评	`p.quote span.inq`	`.quote .inq`
详情链接	`div.hd a` 的 `href`	`.hd a::attr(href)`
下一页	`div.paginator span.next a`	`.paginator .next a`

先把这七个字段找齐，再写循环。不要一上来就写“抓整页”；先确认每个字段都有稳定的落点，尤其是短评：不是每部电影都有。

3.6 CSS 选择器：从入门到够用

CSS 选择器是爬虫最常用的定位语言，BeautifulSoup、Scrapy、Playwright 都支持。先掌握下面十几个模式，足以覆盖豆瓣 Top250 以及大多数经济数据页面。

表 3-4 豆瓣 Top250 常用 CSS 选择器

目标	选择器	说明
所有电影条目	`ol.grid_view > li`	直接子节点，最稳
单个卡片	`.item`	类名定位
片名	`.hd a .title`	先找标题链接，再取 span
评分	`.rating_num`	类名直接对应评分
短评	`.quote .inq`	短评可能为空
评分属性	`[property="v:average"]`	属性等于匹配
详情链接	`a[href*="/subject/"]`	href 包含 subject
第二部电影	`li:nth-child(2)`	按位置，容易随页面变化
下一页	`.paginator .next a`	分页容器里的 next
同时匹配	`.title, .rating_num`	逗号表示并集

属性选择器还有三种常用写法：`^=` 开头匹配、`$=` 结尾匹配、`*=` 包含匹配。

代码 3-4 属性选择器的三种匹配方式

```
a[href^="https://movie.douban.com/subject/"]  /* 以详情页开头 */
a[href$="/1292052/"]                          /* 以电影 ID 结尾 */
a[href*="/subject/"]                          /* 包含 subject */
```

在浏览器控制台里可以直接验证选择器。Chrome/Edge 的 Elements 面板支持 `Ctrl+F` 搜索，Console 里可以用 `document.querySelectorAll()` 计数：

代码 3-5 在豆瓣页面验证选择器

```
document.querySelectorAll("ol.grid_view > li").length
document.querySelector(".hd a .title").innerText
document.querySelector(".paginator .next a").href
```

Scrapy/parsel 的扩展语法

- `::text` 取文本、`::attr(href)` 取属性，是 Scrapy/parsel 的扩展，不是标准 CSS，浏览器控制台不认。
- BeautifulSoup 用 `.get_text()` 和 `tag["href"]`；Playwright 用 `.inner_text()` 和 `.get_attribute("href")`。

3.7 开发者工具实战：在豆瓣页面上找选择器

下面七步是每次抓新页面都值得重复的流程。熟练之后，通常 2—3 分钟就能判断该不该写爬虫。

1. 打开 `https://movie.douban.com/top250`，按 `F12`，进入 Elements 面板。
2. 用左上角箭头选中“肖申克的救赎”这条，观察高亮的 `li`、`.item`、`.hd`、`.bd`。
3. 在 Elements 里按 `Ctrl+F`，搜索“希望让人自由”，确认短评在 `.quote .inq` 里。
4. 在 Console 里运行 `document.querySelectorAll("ol.grid_view > li").length`，应为 25。
5. 切到 Network 面板，筛选 `Fetch/XHR`，刷新页面；豆瓣榜单数据通常不在 XHR 里，而在首次 HTML 中。
6. 按 `Ctrl+U` 查看网页源代码，搜索“肖申克的救赎”；能搜到，说明是静态渲染。
7. 右键节点 → Copy → Copy selector，得到选择器后再简化，不要直接用于生产。

复制选择器的陷阱

- 浏览器复制的通常是又长又脆的绝对路径，可能包含 `nth-child` 和自动生成的 class。
- 优先把它简化为 `ol.grid_view > li`、`.hd a .title`、`.rating_num` 这类稳定结构。

3.8 静态页面与动态页面：豆瓣属于哪一类

静态页面：服务器返回的 HTML 里已经有数据，`requests.get(...).text` 就能搜到。动态页面：数据由 JavaScript 在浏览器里再请求一次，第一次 HTML 里只有空容器。豆瓣 Top250 属于前者：榜单直接写在 HTML 的 `ol.grid_view` 里。

表 3-5 豆瓣 Top250 的静态特征

判断方式	豆瓣 Top250（静态）	典型动态页面
View Source（Ctrl+U）	能搜到片名和评分	搜不到，只有空 div/加载提示
Elements 面板	有数据	有数据（JS 执行后）
Network 的 XHR/Fetch	有广告/统计请求，但榜单数据在 HTML 里	能看到返回榜单 JSON 的请求
禁用 JavaScript 后刷新	榜单仍在	数据消失或显示加载失败
`requests.get().text`	包含 `ol.grid_view` 和片名	不包含数据

如果确认是动态页面，先不要急着上 Playwright。打开 Network → Fetch/XHR，找到返回数据的请求，看它的 URL、参数、请求头和响应。如果这个接口不需要登录、没有一次性签名，往往可以直接用 `requests` 请求 JSON，比浏览器自动化快得多，也稳定得多。

3.9 编码、实体与文本清洗

中文网页常见的编码是 UTF-8、GBK、GB18030。豆瓣页面使用 UTF-8；编码判断错误时，你会看到“锟斤拷”或一堆问号。优先相信响应头或 `<meta charset>`；两者都没有时，再用 `apparent_encoding` 猜测。

代码 3-6 处理豆瓣页面的编码

```
resp = requests.get(url, timeout=30)
resp.raise_for_status()

if not resp.encoding or resp.encoding.lower() == "iso-8859-1":
    resp.encoding = resp.apparent_encoding

# 豆瓣页面通常是 UTF-8，显式设置更稳：
resp.encoding = "utf-8"
html = resp.text
```

HTML 实体（如 ` `、`&`）会被解析器自动转换；空白字符、全角空格、换行需要自己清理。评分是小数，评价人数带“人评价”，排名是整数——这些都要在解析时转成正确的类型。

代码 3-7 清洗带单位的数字

```
import re

def parse_number(text):
    if text is None:
        return None
    s = str(text).replace(",", "").replace(", ", "").strip()
    m = re.search(r"-?\d+(?:\.\d+)?", s)
    return float(m.group()) if m else None

parse_number("3353799 人评价")   # 3353799.0
parse_number("9.7")            # 9.7
```

注意：`parse_number` 只取数字，不保留单位。对于“亿元”“%”“万人”这类字段，单位要单独存一列，否则后续合并数据时会出错。

3.10 选择器的稳定性分级

表 3-6 选择器稳定性分级

稳定性	写法	建议
高	`ol.grid_view > li`、`[property="v:average"]`	首选：语义明确，页面改版通常保留
中	`.item .info .hd a`	可用：依赖业务 class，需回归测试
中低	`div.item div.info`	结构稳定时可用：注意卡片顺序
低	`li:nth-child(3) > div > a`	避免：插入一个广告就失效
很低	`div.css-1x2y3z`	避免：前端构建工具生成的哈希类名

一个实用策略是写“主选择器 + 备选选择器”：主选择器取不到时，尝试备选，并在日志里记录降级。但不要用大量 `try/except` 掩盖结构变化；空结果应该触发告警。

3.11 练习：在豆瓣 Top250 上找选择器

打开 `https://movie.douban.com/top250`，完成下面五件事。这个页面是静态的，适合练手。

- 找到所有电影条目的容器，写出选择器，并在 Console 里数出 25 个。
- 在条目内找到排名、片名、评分、短评，分别写出选择器。
- 找到“下一页”链接，取出它的 `href`，观察 `start` 参数如何变化。
- 按 `Ctrl+U` 查看源代码，确认数据在第一次响应里；把选择器写进一个 Python 列表。

HTML 是一棵树，选择器是走到节点的路径。下一章开始写代码：用 `requests` 拿豆瓣 HTML，用 `BeautifulSoup` 走这棵树。

第 4 章

requests + BeautifulSoup：豆瓣 Top250

这是最轻、最直接的一层：一个请求、一棵树、十页榜单。适合单站、少量页面、一次性任务。

本章目标

- 搭好 Python 环境并发出第一个豆瓣请求
- 用 BeautifulSoup 抽取排名、片名、评分、短评
- 处理十页翻页、限速、保存和常见报错

豆瓣 Top250 有 10 页，每页 25 部电影，共 250 条。页面是静态 HTML，字段规整，正好用来练习“请求—解析—翻页—校验—保存”的完整闭环。真实研究数据往往更复杂，但这一套流程是通用的。

4.1 安装与项目目录

先创建虚拟环境，避免不同项目的依赖互相污染。下面的命令在 PowerShell 中执行。

代码 4-1 创建虚拟环境并安装依赖

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install requests beautifulsoup4 lxml pandas
```

建议的项目结构如下。`data/raw` 放原始 HTML，`data/interim` 放清洗中间结果，`data/processed` 放分析用数据；三者不要混在一起。

代码 4-2 豆瓣 Top250 项目的目录

```
douban_top250/
├─ data/
│  ├─ raw/          # 原始 HTML/JSON，不覆盖
│  ├─ interim/      # 清洗中的 Parquet/CSV
│  └─ processed/    # 分析用数据
├─ scripts/
│  └─ fetch_douban.py
├─ tests/
│  └─ fixtures/     # 保存的 HTML 样本
├─ requirements.txt
└─ README.md
```

4.2 第一个请求：拿到豆瓣的 HTML

先只做一件事：拿到响应，确认状态码、编码和正文。不要一开始就写解析。

代码 4-3 请求豆瓣 Top250 第一页

```
import requests

url = "https://movie.douban.com/top250"
params = {"start": 0, "filter": ""}
resp = requests.get(url, params=params, timeout=30)

print("status:", resp.status_code)
print("url:", resp.url)
print("content-type:", resp.headers.get("content-type"))
print(resp.text[:300])
```

`resp.text` 是解码后的字符串，`resp.content` 是原始字节。中文乱码时，先检查 `resp.encoding`；需要按字节处理（如下载海报）时用 `content`。`timeout` 必须设置，否则网络异常时程序可能一直挂着。

代码 4-4 让错误尽早暴露

```
resp.raise_for_status()   # 4xx/5xx 直接抛异常，避免把错误页当数据
print(resp.encoding)
print(len(resp.content), "bytes")
print("grid_view" in resp.text)   # 确认拿到榜单容器
```

4.3 会话、请求头、超时、重试与限速

用 `requests.Session()` 复用连接和 Cookie，比每次新建请求更稳。豆瓣对高频访问比较敏感，本节示例使用普通浏览器 UA 读取公开页面，并设置 5 秒间隔；**不要**用它伪装登录、绕过风控或高频抓取。

代码 4-5 带请求头的 Session

```
import requests

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept-Language": "zh-CN,zh;q=0.9",
}
session = requests.Session()
session.headers.update(HEADERS)
```

关于 UA 的边界

- 这里使用浏览器 UA 只是为了让公开页面正常返回，不是伪装身份或绕过登录。
- 如果豆瓣返回 403、验证码或要求登录，应停止采集，而不是换代理、换账号继续。

重试只解决“临时性失败”，不能解决“没有权限”。下面的配置对 429 和 5xx 做有限退避，最多 3 次；如果仍然失败，应该记录并停止，而不是无限重试。

代码 4-6　有限重试与超时

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

retry = Retry(
    total=3,
    backoff_factor=1.0,
    status_forcelist=[429, 500, 502, 503, 504],
    allowed_methods=["GET"],
)
session.mount("https://", HTTPAdapter(max_retries=retry))

resp = session.get(url, params={"start": 0}, timeout=30)
resp.raise_for_status()
```

访问前先看 robots.txt。豆瓣的 robots.txt 没有禁止 `/top250`，但有一条被注释掉的 `Crawl-delay: 5`；本书按 5 秒执行。`RobotFileParser` 可以帮你做基本检查，真正的授权仍要看服务条款和许可。

代码 4-7　检查豆瓣 robots.txt

```
from urllib.robotparser import RobotFileParser

rp = RobotFileParser()
rp.set_url("https://movie.douban.com/robots.txt")
rp.read()
print(rp.can_fetch("*", "https://movie.douban.com/top250"))
```

请求之间加一个固定间隔，是最简单的限速；对豆瓣，建议从 5 秒起，不要并发轰炸。

代码 4-8　串行请求与 5 秒间隔

```
import time

for start in range(0, 250, 25):
    resp = session.get(url, params={"start": start}, timeout=30)
    resp.raise_for_status()
    process(resp.text)
    time.sleep(5)
```

4.4 BeautifulSoup 的基本操作

`BeautifulSoup` 把 HTML 字符串解析成树。解析器选 `lxml`（快、容错好），没有安装时可以用 Python 自带的 `html.parser`。

代码 4-9 BeautifulSoup 找到 25 个电影条目

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(resp.text, "lxml")

print(soup.title.string)
cards = soup.select("ol.grid_view > li")
print("cards:", len(cards))

first = cards[0]
print(first.select_one(".hd a .title").get_text(strip=True))
print(first.select_one(".rating_num").get_text(strip=True))
```

表 4-1 BeautifulSoup 常用方法

方法	返回	用途
`soup.find("ol")`	第一个匹配的 Tag 或 `None`	已知只有一个榜单容器
`soup.find_all("li")`	列表	所有列表项
`soup.select("ol.grid_view > li")`	列表（CSS 选择器）	最常用
`soup.select_one(".item")`	第一个或 `None`	卡片内取字段
`tag.get_text(" ", strip=True)`	字符串	合并子节点文本并去空白
`tag.get("href")` / `tag["href"]`	属性值	取详情页链接

`select_one()` 找不到时返回 `None`，继续调用 `.get_text()` 会报 `'NoneType' object has no attribute ...`。短评不是每部电影都有，必须显式判断。

代码 4-10 对可能缺失的短评做判断

```
card = cards[0]
quote_node = card.select_one(".quote .inq")
quote = quote_node.get_text(strip=True) if quote_node else ""
print(quote)   # 希望让人自由。
```

4.5 抽取一个电影条目：排名、片名、评分、短评

最稳的做法是“先定位卡片，再在卡片内取字段”。不要在整页上写一堆全局选择器，否则很容易把导航栏、广告或页脚的文字混进来。

代码 4-11 从一个卡片里抽取五个字段

```
def parse_card(card):
    rank_node = card.select_one(".pic em")
    title_node = card.select_one(".hd a .title")
    rating_node = card.select_one(".rating_num")
    quote_node = card.select_one(".quote .inq")
    url_node = card.select_one(".hd a")

    return {
        "rank": int(rank_node.get_text(strip=True)) if rank_node else None,
        "title": title_node.get_text(strip=True) if title_node else "",
        "rating": float(rating_node.get_text(strip=True)) if rating_node else None,
        "quote": quote_node.get_text(strip=True) if quote_node else "",
        "url": url_node.get("href") if url_node else "",
    }
```

如果页面是表格而不是卡片，思路类似：先定位 `table`，再取 `thead th` 作为列名，最后逐行取 `tbody tr` 的 `td`。表格的合并单元格（`rowspan`/`colspan`）会让列数错位，先用 `pandas.read_html()` 看一眼结果，再决定是否手写展开逻辑。

字段缺失是常态

- 豆瓣条目不一定有短评，也不一定有“可播放”标记；不要假设每个字段都存在。
- 对缺失字段保留空字符串或 `None`，并在数据字典里写清缺失规则。

4.6 列表页 → 详情页：先拿链接，不要急着进详情

Top250 的排名、片名、评分、短评都在列表页，通常不需要进详情页。如果你确实需要导演、时长、类型等详情字段，先从列表页收集 `subject` 链接，再对详情页单独限速抓取。

代码 4-12 从榜单卡片收集详情页链接

```
from urllib.parse import urljoin

links = []
for card in soup.select("ol.grid_view > li"):
    a = card.select_one(".hd a")
    if a and a.get("href"):
        links.append(urljoin(url, a["href"]))

print(len(links), links[:2])
```

`urljoin` 负责把相对链接转成绝对 URL。详情页解析时，先保存 `response.url`，它是 provenance 的一部分；以后数据对不上，可以回到原页面核对。

4.7 翻页：start 参数每次加 25

豆瓣 Top250 的分页参数是 `start`：第一页 `0`，第二页 `25`，一直到 `225`。最稳妥的循环要同时具备三个停止条件：没有下一页、达到最大页数、URL 重复。

代码 4-13 用 start 参数翻完 10 页

```
from urllib.parse import urljoin

url = "https://movie.douban.com/top250"
seen = set()
all_rows = []

for start in range(0, 250, 25):
    page_url = f"{url}?start={start}&filter="
    if page_url in seen:
        break
    seen.add(page_url)

    resp = session.get(page_url, timeout=30)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "lxml")
    all_rows.extend(parse_card(li) for li in soup.select("ol.grid_view > li"))
    time.sleep(5)
```

也可以从页面里的 `.paginator .next a` 取下一页链接；两种方式等价。前者更可控，后者更贴近页面真实结构。无论哪种，都要记录已经抓过的 URL。

4.8 保存：CSV、JSONL 与断点

小规模数据用 CSV 最方便；需要保留嵌套字段或追加写入时用 JSONL；数据量大、要做类型分析时用 Parquet。给 Excel 打开的 CSV 建议用 `utf-8-sig`，否则中文可能乱码。

代码 4-14 写入豆瓣 Top250 的 CSV

```
import csv

fields = ["rank", "title", "rating", "quote", "url"]
with open("data/processed/douban_top250.csv", "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(all_rows)
```

代码 4-15 追加写入 JSONL

```
import json

with open("data/raw/douban_top250.jsonl", "a", encoding="utf-8") as f:
    for row in all_rows:
        f.write(json.dumps(row, ensure_ascii=False) + "\n")
```

断点续跑的最低要求是：每处理完一页就落盘，并记录已完成的 `start`。程序中断后，从已完成集合里恢复，不要从头再抓一遍。

代码 4-16　用 done 集合实现简单断点

```python
import json, os

def load_done(path="data/raw/douban_done.json"):
    if not os.path.exists(path):
        return set()
    with open(path, encoding="utf-8") as f:
        return set(json.load(f))

def save_done(done, path="data/raw/douban_done.json"):
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(sorted(done), f, ensure_ascii=False, indent=2)
    os.replace(tmp, path)
```

保存前先做三件事

- 打印 `len(rows)`，确认是否为 250；不是就先别写文件。
- 检查 `rank` 是否从 1 连续到 250，有没有重复或缺口。
- 检查 `rating` 是否在 0—10，`url` 是否唯一且包含 `/subject/`。

表 4-5　三种保存格式的取舍

格式	适合	注意
CSV	250 条一次性交付、Excel 查看	用 `utf-8-sig`；没有类型
JSONL	逐页追加、保留嵌套字段	一行一条；适合日志式快照
Parquet	多次快照、类型分析、DuckDB 查询	需要 pyarrow；不适合直接发邮件

豆瓣 Top250 只有 250 条，CSV 足够；如果你打算每周抓一次做比较，建议按日期保存 JSONL 或 Parquet，再用 DuckDB 合并。

4.9 完整案例：十页、250 条、一次跑通

下面把前面的片段拼成一个可运行脚本：抓取 10 页，解析排名、片名、评分、短评和链接，保存 CSV，并打印总条数。建议先跑第一页，确认字段后再全量。

代码 4-17 豆瓣 Top250 完整抓取脚本

```
import csv
import time

import requests
from bs4 import BeautifulSoup

BASE = "https://movie.douban.com/top250"
HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept-Language": "zh-CN,zh;q=0.9",
}

session = requests.Session()
session.headers.update(HEADERS)

rows = []
for start in range(0, 250, 25):
    resp = session.get(BASE, params={"start": start}, timeout=30)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "lxml")

    for card in soup.select("ol.grid_view > li"):
        rank_node = card.select_one(".pic em")
        title_node = card.select_one(".hd a .title")
        rating_node = card.select_one(".rating_num")
        quote_node = card.select_one(".quote .inq")
        url_node = card.select_one(".hd a")
        rows.append({
            "rank": int(rank_node.get_text(strip=True)) if rank_node else None,
            "title": title_node.get_text(strip=True) if title_node else "",
            "rating": float(rating_node.get_text(strip=True)) if rating_node else None,
            "quote": quote_node.get_text(strip=True) if quote_node else "",
            "url": url_node.get("href") if url_node else "",
        })

    print("page", start // 25 + 1, "rows", len(rows))
    time.sleep(5)

with open("douban_top250.csv", "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.DictWriter(
        f, fieldnames=["rank", "title", "rating", "quote", "url"]
    )
    writer.writeheader()
    writer.writerows(rows)

print("total:", len(rows))
```

运行时观察三件事：页数是否为 10、总数是否为 250、排名是否从 1 连续到 250。如果页数对但条数少，通常是选择器写窄了；如果条数多，可能是翻页循环重复。

4.10 豆瓣案例的边界：什么时候不要用 BeautifulSoup

豆瓣 Top250 是公开榜单，适合练习。但它不是经济学数据源；正式研究应优先使用官方 API、统计年鉴、机构订阅或商业数据库。下表中的场景，值得先停下来判断。

三个“先别写”的信号

- 页面数据在 View Source 里搜不到——先找 XHR/JSON，或直接用 Playwright。
- 站点有登录、验证码、付费墙——先确认授权，不要绕过。
- 数据有官方批量文件/API——优先用官方渠道，网页只做补充。

表 4-2 豆瓣案例与其他数据源的边界

目标	更合适的做法
世界银行/FRED 宏观指标	回到第 2 章，用官方 API
国家统计局年度数据	优先用官网导出/年鉴，网页只做补充
企业公告与财报	交易所/巨潮的公告 PDF，或商业数据库
豆瓣 Top250	低频、公开字段、教学用途；不批量抓用户信息

4.11 常见错误与排查

表 4-3 豆瓣静态抓取的常见错误

现象	可能原因	处理
403 Forbidden	豆瓣限制、UA 被拒、访问过快	停止并降速；不要换代理硬绕
429 Too Many Requests	请求太快	读 `Retry-After`，退避，降低并发
Timeout	网络慢、站点限流	增大 timeout、重试、减少单页体积
中文乱码	编码判断错误	显式设置 `resp.encoding = "utf-8"`
`NoneType` 报错	短评/评分节点不存在	保存 HTML，回浏览器验证选择器
结果为空列表	选择器过窄 / 页面改版	检查 `ol.grid_view` 是否存在
重复行	翻页循环重复	用 `rank` 或 `url` 去重
CSV 用 Excel 打开乱码	没有 BOM	写 CSV 时用 `utf-8-sig`
评分变成字符串	没有转 float	清洗后 `float()` 或 `pd.to_numeric`

排查顺序建议固定下来：先用浏览器打开同一个 URL；再用 `requests` 打印状态码和 `text[:500]`；然后把 HTML 保存到本地，用同一个选择器在本地文件上测试；最后才怀疑翻页、编码或反爬。

4.12 检查清单

表 4-4 requests + BeautifulSoup 检查清单

检查项	通过标准
权限	已看 robots.txt、服务条款；没有绕过登录/付费墙
请求	有 UA、超时、重试、5 秒间隔；串行
解析	选择器在保存的 HTML 上可复现；缺失短评有默认值
翻页	10 页、start 从 0 到 225；有停止条件
保存	原始 HTML 与清洗数据分开；CSV 编码正确
断点	中断后不会从头再来；已完成 start 有记录
校验	250 条；排名 1—250；评分 0—10；URL 唯一

第 5 章

Scrapy：豆瓣 Top250 的规模化抓取

当页面从十页变成几千页，你需要的不再是循环，而是队列、去重、重试、并发和管道。

本章目标

- 创建并组织一个豆瓣 Top250 Scrapy 项目
- 写 Spider、Item、Pipeline 和翻页逻辑
- 配置限速、断点续爬、导出与日志

豆瓣 Top250 只有 10 页，用第 4 章的脚本足够。但如果你要抓的是“豆瓣 Top250 + 历年榜单 + 多地区版本”，或者每周更新一次，就需要 Scrapy 这样的框架来管理请求队列、去重、断点和并发。本章仍以 Top250 为例，把同一份数据用 Scrapy 重写一遍，方便你对比两种方案。

5.1 什么时候该用 Scrapy

表 5-1 两种方案的边界

维度	requests + BS4	Scrapy
页面规模	几十到几百页	几千页以上，或多站点
运行方式	一次性脚本	周期性、长期运行
并发	自己写线程/异步	内置并发与自动限速
去重/断点	自己维护 seen 集合	内置 dupefilter、JOBDIR
数据管道	函数里清洗	Item Pipeline 统一处理
学习成本	低	中高

不要为了用框架而用框架

- 如果只抓豆瓣 Top250 的 250 条，Scrapy 的项目结构反而增加负担。先用第 4 章的方法把逻辑跑通，确认需要规模和周期性后，再迁移到 Scrapy。

5.2 安装与创建项目

代码 5-1 创建豆瓣 Top250 的 Scrapy 项目

```
pip install scrapy
scrapy startproject douban_top250
cd douban_top250
scrapy genspider top250 movie.douban.com
```

代码 5-2 Scrapy 项目的默认结构

```
douban_top250/
├─ scrapy.cfg
├─ douban_top250/
│  ├─ __init__.py
│  ├─ items.py
│  ├─ middlewares.py
│  ├─ pipelines.py
│  ├─ settings.py
│  └─ spiders/
│     └─ top250.py
└─ data/
```

`scrapy.cfg` 是项目入口；`settings.py` 放全局配置；`spiders/` 放抓取逻辑；`items.py` 定义字段；`pipelines.py` 做清洗、去重和入库。不要把所有逻辑都塞进 Spider。

5.3 Spider 的基本结构

一个 Spider 至少要有 `name`、起始 URL 和 `parse` 回调。`parse` 收到的是 `Response` 对象，你可以从中 `yield` 三种东西：字典/Item（数据）、新的 Request（继续抓）、`None`（忽略）。

代码 5-3 一个带翻页的豆瓣 Top250 Spider

```python
import scrapy

class Top250Spider(scrapy.Spider):
    name = "top250"
    allowed_domains = ["movie.douban.com"]
    start_urls = ["https://movie.douban.com/top250?start=0&filter="]

    def parse(self, response):
        for card in response.css("ol.grid_view > li"):
            yield {
                "rank": card.css(".pic em::text").get(default="").strip(),
                "title": card.css(".hd a .title::text").get(default="").strip(),
                "rating": card.css(".rating_num::text").get(default="").strip(),
                "quote": card.css(".quote .inq::text").get(default="").strip(),
                "url": card.css(".hd a::attr(href)").get(),
            }

        next_page = response.css(".paginator .next a::attr(href)").get()
        if next_page:
            yield response.follow(next_page, callback=self.parse)
```

`response.follow()` 会自动处理相对 URL，并继承当前请求的 meta；比手动 `urljoin` 更适合 Scrapy。`allowed_domains` 限制可抓域名，防止误爬到外站；豆瓣详情页在 `movie.douban.com`，不需要额外加域名。

5.4 Scrapy 的选择器

Scrapy 使用 parsel，语法与 BeautifulSoup 的 `select` 类似，但多了 `::text` 和 `::attr()`。`get()` 取第一个，`getall()` 取全部，`get(default="")` 避免 `None`。

表 5-2 Scrapy/parsel 的豆瓣选择器

目标	写法	返回
所有电影条目	`response.css("ol.grid_view > li")`	SelectorList
片名	`card.css(".hd a .title::text").get()`	字符串或 None
评分	`card.css(".rating_num::text").get()`	字符串或 None
短评	`card.css(".quote .inq::text").get()`	字符串或 None
详情链接	`card.css(".hd a::attr(href)").get()`	字符串或 None
下一页	`response.css(".paginator .next a::attr(href)").get()`	字符串或 None
XPath 版本	`response.xpath("//ol[@class='grid_view']/li")`	SelectorList

`::text` 只取直接文本节点；如果文字在子标签里，用 `*::text` 或更具体的子选择器。评分、排名建议在 Pipeline 里统一转类型，不要在 Spider 里写太多 `replace`。

5.5 Item 与 Pipeline

Item 定义字段，Pipeline 负责清洗、去重、校验和存储。把“抓取”和“加工”分开，以后换存储或加字段时，不需要改 Spider。

代码 5-4 定义豆瓣 Top250 的 Item 字段

```python
import scrapy

class Top250Item(scrapy.Item):
    rank = scrapy.Field()
    title = scrapy.Field()
    rating = scrapy.Field()
    quote = scrapy.Field()
    url = scrapy.Field()
```

代码 5-5 在 Pipeline 中清洗排名与评分

```python
class CleanTop250Pipeline:
    def process_item(self, item, spider):
        rank = item.get("rank", "").strip()
        rating = item.get("rating", "").strip()
        item["rank"] = int(rank) if rank.isdigit() else None
        item["rating"] = float(rating) if rating else None
        item["title"] = " ".join(item.get("title", "").split())
        return item
```

代码 5-6 启用 Pipeline

```
ITEM_PIPELINES = {
    "douban_top250.pipelines.CleanTop250Pipeline": 300,
}
```

Pipeline 的 `process_item` 必须返回 Item 或抛出 `DropItem`。如果要写数据库，建议在 Pipeline 里做批量插入，而不是每一条都开一次连接。

5.6 翻页、详情页与请求控制

翻页有两种常见写法：在列表页 `yield response.follow(next_page, callback=self.parse)`，或者用 `start_requests` 自己生成 10 个 `start` URL。前者适合“下一页”链接，后者适合固定页数。

代码 5-7 用 start 参数生成 10 个起始请求

```
def start_requests(self):
    for start in range(0, 250, 25):
        url = f"https://movie.douban.com/top250?start={start}&filter="
        yield scrapy.Request(url, callback=self.parse)
```

如果详情页需要登录态，可以在 `start_requests` 里设置 `cookies` 或 `headers`；但不要把账号密码写进代码。豆瓣 Top250 本身不需要登录，建议不要为了“多抓一点”去登录或绕过限制。

回调函数写错，什么都不会发生

- `callback=self.parse_detial` 这种拼写错误不会报错，只会让请求被丢弃或进入默认解析。
- 用 `scrapy crawl top250 -L INFO` 看日志中的 `Crawled` 和 `Scraped` 数量；数量为 0 时先查回调名。

5.7 必须理解的几个 settings

表 5-3 豆瓣项目的常用 Scrapy 配置

配置	建议值	作用
`ROBOTSTXT_OBEY`	`True`	遵守 robots.txt；不要为了速度关掉
`DOWNLOAD_DELAY`	`5` 起	豆瓣建议的低频访问；每页之间至少 5 秒
`CONCURRENT_REQUESTS`	`1` 起	先串行；确认稳定后再谈并发
`AUTOTHROTTLE_ENABLED`	`True`	根据响应延迟自动调整速度
`RETRY_TIMES`	`3`	临时失败重试次数
`USER_AGENT`	浏览器 UA	让公开页面正常返回；不要伪装登录
`FEEDS`	JSONL/CSV	导出路径与格式

配置	建议值	作用
LOG_LEVEL	INFO	排错时用 DEBUG，生产用 INFO

代码 5-8　一份保守的豆瓣 settings 片段

```
ROBOTSTXT_OBEY = True
DOWNLOAD_DELAY = 5
CONCURRENT_REQUESTS = 1
AUTOTHROTTLE_ENABLED = True
RETRY_TIMES = 3
USER_AGENT = (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/124.0.0.0 Safari/537.36"
)
FEEDS = {
    "data/douban_top250.jsonl": {
        "format": "jsonl", "encoding": "utf8", "overwrite": True
    },
}
```

5.8 运行、导出与日志

代码 5-9　运行豆瓣 Spider 的常用命令

```
# 覆盖写入 JSONL
scrapy crawl top250 -O data/douban_top250.jsonl

# 追加写入 CSV
scrapy crawl top250 -o data/douban_top250.csv

# 指定日志级别
scrapy crawl top250 -L INFO
```

日志里重点看四个数字：item_scraped_count、response_received_count、retry/count、httperror/403_count。如果请求数正常但 item 数为 0，通常是选择器或回调问题；如果 403 很多，先停下来，不要换代理。

5.9 增量抓取与断点续爬

Scrapy 默认会对 URL 去重，但“URL 没变、榜单变了”的页面仍会被跳过。需要增量更新时，用数据库唯一键 + 内容哈希，或者在请求上带 dont_filter=True 并自己判断。

代码 5-10　用 JOBDIR 断点续爬

```
# 第一次运行后保留状态目录
scrapy crawl top250 -s JOBDIR=state/top250

# 中断后再次执行同一命令，会从断点继续
scrapy crawl top250 -s JOBDIR=state/top250
```

`JOBDIR` 保存请求队列和去重指纹，适合“抓一半断了”的场景。长期增量更新更适合把结果写入数据库，用 `(snapshot_date, rank)` 或 `(snapshot_date, url)` 做唯一约束，再用 `last_seen`、`content_hash` 判断是否更新。

5.10 中间件、代理与边界

下载中间件可以改请求头、加代理、处理异常。代理不是“绕过封锁”的许可证：如果站点明确拒绝，换 IP 继续抓既不礼貌，也可能违反条款。代理只应在“有权访问、但网络位置受限”的场景下使用。

代码 5-11 一个最小的代理中间件（仅示意）

```
class ProxyMiddleware:
    def process_request(self, request, spider):
        request.meta["proxy"] = "http://127.0.0.1:7890"
```

先问“我有没有权利”，再问“我能不能绕过”

- 被封禁、要求登录、出现验证码，都是站点在表达访问边界。此时应改用 API、申请授权或购买数据。
- 本指南不提供验证码识别、指纹伪造、账号池等规避手段。

5.11 Scrapy + Playwright：豆瓣其实不需要

豆瓣 Top250 是静态页面，用 Scrapy 的 HTTP 下载器就够了。下面保留 `scrapy-playwright` 的配置，是为了让你在遇到真正依赖 JavaScript 的页面时知道怎么接；不要对豆瓣全站开浏览器。

代码 5-12 安装 scrapy-playwright

```
pip install scrapy-playwright
playwright install chromium
```

代码 5-13 settings 中的 Playwright 配置

```
DOWNLOAD_HANDLERS = {
    "http": "scrapy_playwright.handler.ScrapyPlaywrightDownloadHandler",
    "https": "scrapy_playwright.handler.ScrapyPlaywrightDownloadHandler",
}
TWISTED_REACTOR = "twisted.internet.asyncioreactor.AsyncioSelectorReactor"
```

代码 5-14 只对需要的请求启用浏览器

```
from scrapy_playwright.page import PageMethod

yield scrapy.Request(
    "https://movie.douban.com/top250?start=0&filter=",
    meta={
        "playwright": True,
        "playwright_page_methods": [
            PageMethod("wait_for_selector", "ol.grid_view > li"),
        ],
    },
)
```

5.12 常见坑

- `allowed_domains` **太严**：详情页在另一个域名时，请求会被过滤。
- **回调名拼错**：不会报错，但不会有 item；看日志的 `Crawled` 数。
- **200 但空结果**：选择器只匹配了空容器，或页面被替换成验证页。
- **相对 URL 没处理**：用 `response.follow` 或 `urljoin`，不要手拼字符串。
- **并发太高**：豆瓣 403/封禁；先降到 1 并发、5 秒间隔，再谈其他。
- **重复抓取**：分页循环没有停止条件，或详情页 URL 带随机参数。
- **Pipeline 抛异常**：单条数据会丢，检查日志里的 `DropItem` 和 traceback。

5.13 检查清单

表 5-4 Scrapy 项目检查清单

检查项	通过标准
结构	Spider 只负责抓取，清洗在 Pipeline
选择器	在保存的 HTML 上可复现；有默认值
翻页	10 页；start 从 0 到 225；不重复
配置	robots、5 秒延迟、1 并发、重试、UA 都已设置
断点	JOBDIR 或数据库幂等可恢复
导出	JSONL/CSV 路径明确，字段完整
日志	能看出请求数、item 数、403/429 数
合规	没有绕过登录、付费墙、验证码

第 6 章

Playwright：用浏览器打开豆瓣 Top250

当数据必须等浏览器执行完脚本才出现，Playwright 是最后的手段，也是最接近“真人操作”的方案。

本章目标

- 安装并启动 Chromium
- 用 locator 等待、点击、截图、翻页
- 用动态页对照 requests 与 Playwright 的差异

Playwright 驱动真实浏览器，能执行 JavaScript、处理点击和登录、下载文件。它比 `requests` 慢得多，也更占资源，所以只应该用在“确实需要浏览器”的页面。豆瓣 Top250 其实不需要，但它是很好的“打开页面、等待元素、点击下一页、抽取卡片”的练习对象；本章最后再用一个真正动态渲染的页面做对照。

6.1 什么时候用 Playwright

表 6-1 Playwright 的适用边界

场景	用 Playwright?	替代方案
数据在第一次 HTML 里（豆瓣 Top250）	不需要	requests + BS4
数据由 XHR 返回 JSON	通常不需要	requests 直接请求接口
需要点击、滚动、登录	需要	—
需要下载文件/附件	需要	找直接下载链接
接口有一次性签名/强风控	可能需要	先确认授权与条款
大规模多页	谨慎	Scrapy + 少量 Playwright 请求

豆瓣案例的结论

- 豆瓣 Top250 是静态 HTML，`requests` 更快、更省资源；本章用 Playwright 是为了演示浏览器自动化，不是因为它更适合这个站点。
- 真实项目里，先判断页面是否需要 JS；不需要就不要开浏览器。

6.2 安装

代码 6-1 安装 Playwright 与 Chromium

```
pip install playwright
playwright install chromium
```

`playwright install` 会下载浏览器，首次安装需要一些时间。Linux 服务器上可能还需要 `playwright install --with-deps chromium` 安装系统依赖。

6.3 第一个脚本：打开豆瓣并等待榜单

代码 6-2 打开豆瓣 Top250 并等待 25 个条目

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto(
        "https://movie.douban.com/top250?start=0&filter=",
        wait_until="domcontentloaded",
        timeout=30_000,
    )
    page.wait_for_selector("ol.grid_view > li")
    print(page.title())
    print(page.locator("ol.grid_view > li").count())
    browser.close()
```

`sync_playwright()` 适合脚本；在异步框架（如 FastAPI、Scrapy）里用 `async_playwright()`。`headless=True` 不显示浏览器窗口；调试时改成 `False`，并加 `slow_mo=300` 看清每一步。

6.4 Locator 与等待

Playwright 的核心是 locator：它描述“怎么找到一个元素”，而不是立刻抓取。操作时自动等待元素可行动，比固定 `sleep` 稳得多。

代码 6-3 用 locator 读取第一名

```
cards = page.locator("ol.grid_view > li")
print(cards.count())          # 25

first = cards.first
print(first.locator(".hd a .title").inner_text())
print(first.locator(".rating_num").inner_text())
print(first.locator(".quote .inq").inner_text())
```

表 6-2 豆瓣案例里的等待与定位

需求	写法	说明
等待榜单出现	`page.wait_for_selector("ol.grid_view > li")`	最常用
等待 URL	`page.wait_for_url("**/top250**")`	跳转后
断言数量	`expect(cards).to_have_count(25)`	需要 `from playwright.sync_api import expect`
按文本找	`page.get_by_text("后页>")`	注意唯一性
取第 2 个	`cards.nth(1)`	0 基索引
取全部文本	`cards.all_inner_texts()`	列表

避免 `page.goto(..., wait_until="networkidle")` 作为默认等待：有轮询、长连接或广告的页面可能永远不“空闲”。更稳的做法是 `domcontentloaded` + 等待具体元素/响应。

6.5 点击“后页”：翻到第二页

豆瓣的分页是普通链接，Playwright 可以直接点击 `.paginator .next a`，等待 URL 或列表更新，再抽取第二页。

代码 6-4 点击后页并抽取第二页

```
page.locator(".paginator .next a").click()
page.wait_for_url("**/top250?start=25**")
page.wait_for_selector("ol.grid_view > li")

print(page.locator("ol.grid_view > li").count())   # 25
print(page.locator(".pic em").first.inner_text())  # 26
```

如果链接文本不稳定，可以用 `page.locator(".paginator .next a").click()`；如果点击后没有跳转，先检查是否有弹窗、登录墙或验证页。

6.6 抽取一页：排名、片名、评分、短评

代码 6-5 用 Playwright 抽取一页的五个字段

```
def parse_page(page):
    rows = []
    for card in page.locator("ol.grid_view > li").all():
        rows.append({
            "rank": card.locator(".pic em").inner_text(),
            "title": card.locator(".hd a .title").first.inner_text(),
            "rating": card.locator(".rating_num").inner_text(),
            "quote": card.locator(".quote .inq").inner_text()
                     if card.locator(".quote .inq").count() else "",
            "url": card.locator(".hd a").first.get_attribute("href"),
        })
    return rows
```

`locator(...).count()` 会立即查询；在动态页面里，先 `wait_for_selector` 再计数，避免拿到 0。短评可能不存在，所以先判断 `count()`。

6.7 截图与调试

- `headless=False` 打开可见浏览器；`slow_mo=300` 放慢动作。
- `page.screenshot(path="douban.png", full_page=True)` 保存整页截图。
- `page.pause()` 在脚本中暂停，配合 `PWDEBUG=1` 使用。
- 失败时先截图，再检查选择器和 URL；不要靠猜。

代码 6-6 保存豆瓣页面截图

```
page.screenshot(path="data/raw/douban_top250_page1.png", full_page=True)
```

6.8 并发与资源

Playwright 的资源开销主要在浏览器进程和页面。不要为每个 URL 启动一个浏览器；正确做法是一个浏览器、少量 page 复用，并用信号量限制并发。豆瓣案例只有 10 页，串行足够。

代码 6-7 复用浏览器与页面

```
browser = p.chromium.launch(headless=True)
context = browser.new_context()
pages = [context.new_page() for _ in range(2)]

for page, start in zip(pages, [0, 25]):
    page.goto(
        f"https://movie.douban.com/top250?start={start}&filter=",
        wait_until="domcontentloaded",
    )
    page.wait_for_selector("ol.grid_view > li")

context.close()
browser.close()
```

每个页面都要设置默认超时（如 30 秒），并在异常时关闭 page，避免内存泄漏。长时间运行时，定期重启浏览器比无限复用更稳。

6.9 与 Scrapy 集成

如果项目主体是 Scrapy，只有少数页面需要浏览器，用 `scrapy-playwright` 按请求开启（见 5.11）。豆瓣 Top250 不需要；如果整个站点都依赖 JS，直接用 Playwright 写脚本更简单。

代码 6-8 scrapy-playwright 的常用配置

```
PLAYWRIGHT_BROWSER_TYPE = "chromium"
PLAYWRIGHT_LAUNCH_OPTIONS = {"headless": True}
PLAYWRIGHT_DEFAULT_NAVIGATION_TIMEOUT = 30_000
```

6.10 完整案例：抓两页并保存

把代码 6-5 的 `parse_page` 函数放在同一个文件里，再运行下面的脚本。

代码 6-9 Playwright 抓取豆瓣前两页

```
import csv
from playwright.sync_api import sync_playwright

rows = []
with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto(
        "https://movie.douban.com/top250?start=0&filter=",
        wait_until="domcontentloaded",
        timeout=30_000,
    )

    for _ in range(2):
        page.wait_for_selector("ol.grid_view > li")
        rows.extend(parse_page(page))
        next_link = page.locator(".paginator .next a")
        if not next_link.count():
            break
        next_link.click()
        page.wait_for_load_state("domcontentloaded")

    browser.close()

with open("douban_top250_playwright.csv", "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.DictWriter(
        f, fieldnames=["rank", "title", "rating", "quote", "url"]
    )
    writer.writeheader()
    writer.writerows(rows)

print("rows:", len(rows))
```

对比一下：用 `requests` 请求同一个 URL，HTML 里已经有 25 个 `li`；这就是为什么说豆瓣不需要浏览器。如果换成一个真正由 JS 渲染的页面，Playwright 才有不可替代的价值。

6.11 补充实例：抓取真正动态渲染的页面

豆瓣 Top250 的榜单写在第一次 HTML 里，所以前面的 Playwright 脚本只是“用浏览器做静态抓取”。为了看清动态页面的差别，这里使用公开教学沙盒 `quotes.toscrape.com/js/`：它的 quote 由 JavaScript 注入，`requests` 看到的是空容器，Playwright 等待后才看到 10 条。

代码 6-10 同一个 URL：requests 看不到，Playwright 能看到

```python
import requests
from bs4 import BeautifulSoup
from playwright.sync_api import sync_playwright

URL = "https://quotes.toscrape.com/js/"

# A. requests：第一次HTML 里没有 quote
html = requests.get(URL, timeout=30).text
soup = BeautifulSoup(html, "lxml")
print("requests:", len(soup.select("div.quote")))   # 0

# B. Playwright：等待 JS 注入后再取
with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto(URL, wait_until="domcontentloaded", timeout=30_000)
    page.wait_for_selector("div.quote")
    rows = []
    for card in page.locator("div.quote").all():
        rows.append({
            "quote": card.locator("span.text").inner_text(),
            "author": card.locator("small.author").inner_text(),
            "tags": ";".join(card.locator("a.tag").all_inner_texts()),
        })
    print("playwright:", len(rows))   # 10
    browser.close()

print(rows[0])
```

表 6-3 静态页与动态页的对照

检查	豆瓣 Top250	动态沙盒
View Source（Ctrl+U）	有 25 个 `li` 和片名	只有空容器/脚本
`requests` + BS4	25 条	0 条
Playwright	25 条	10 条
结论	静态页，优先 requests	动态页，才需要浏览器

动态页的三种处理顺序

- ① 先在 Network 里找返回 JSON 的 XHR；能复现就用 `requests`，不要开浏览器。
- ② 数据写在内联 `<script>` 数组里：可以解析脚本，但结构易变，需固定样本测试。
- ③ 只能靠浏览器渲染、点击、滚动或登录：才用 Playwright。

注意：`quotes.toscrape.com/js/` 是公开教学沙盒，不是经济学数据源；它在这里只用于对照“第一次 HTML 没有数据”的场景。主案例仍然是豆瓣 Top250。

6.12 常见坑与检查清单

表 6-4 Playwright 常见问题

现象	原因	处理
一直等到超时	选择器错/页面没加载/验证页	截图；先等 `body`；检查 URL
元素找到了但点不动	被遮挡/不可见/未启用	滚动到可见；等按钮 enabled
翻页后还是第一页	点击未生效/等待不足	等 URL 或 `.pic em` 变化
服务器上失败、本地成功	字体/无头/网络/反爬	看截图；不要伪造指纹
内存越来越大	page/context 未关闭	限制并发；定期重启浏览器

表 6-5 Playwright 检查清单

检查项	通过标准
必要性	已确认没有可复现的 JSON 接口；豆瓣是静态页
等待	等具体元素/URL，不用固定 sleep
并发	浏览器复用，页面数受限，超时明确
调试	失败时有截图，不靠猜
合规	没有绕过登录、付费墙、验证码或风控

第 7 章

组合方案与工程化

把一次性脚本变成可重复运行的项目：目录、配置、调度、存储、日志和测试。

本章目标

- 为豆瓣 Top250 项目选择主方案
- 搭一个最小但完整的项目骨架
- 让抓取可调度、可监控、可维护

到了这一步，你已经能用四种方式取数。真正决定项目能否长期运行的，不是某个库，而是工程组织：配置是否集中、失败是否可恢复、数据是否有层次、日志是否能定位问题。

7.1 组合决策表

表 7-1 按数据源特征组合方案

数据源特征	首选	备选	不要用
有 API / 批量下载	API + pandas	官方导出	浏览器自动化
静态 HTML，少量页面（豆瓣 10 页）	requests + BS4	pandas.read_html	Scrapy
静态 HTML，多页/周期（豆瓣 + 历年）	Scrapy	requests + 队列	逐页手写循环
JS 渲染，少量页面	Playwright	找 XHR 接口	Scrapy 纯 HTTP
JS 渲染，大规模	Scrapy + 少量 Playwright	Playwright 脚本	全站浏览器
登录后下载	Playwright / 官方 API	机构订阅	绕过登录

7.2 一个可维护的项目骨架

代码 7-1 豆瓣 Top250 项目的推荐目录

```
douban_top250/
├─ config/
│  └─ sources.yaml       # 数据源登记
├─ src/
│  ├─ api/               # API 取数（预留）
│  ├─ scrapers/          # requests/Scrapy/Playwright
│  ├─ parsers/           # HTML 解析
│  └─ pipelines/         # 清洗、校验、入库
├─ data/
│  ├─ raw/               # 原始 HTML，不覆盖
│  ├─ interim/           # 中间结果
│  └─ processed/         # 分析用数据
├─ logs/
├─ tests/
│  └─ fixtures/          # 固定 HTML 样本
├─ requirements.txt
└─ README.md
```

`raw` 只追加、不修改；`interim` 可以重算；`processed` 是分析直接使用的数据。把这三层分开，任何一步出错都能回滚，而不是从头再抓。

7.3 配置与密钥

把 URL、起始页、输出路径写进配置文件；如果以后接入需要 key 的 API，把 key 放进环境变量。豆瓣 Top250 不需要 key，但项目结构应该为以后扩展留位置。

代码 7-2 用 YAML 登记豆瓣数据源

```yaml
sources:
  - name: douban_top250
    type: static
    start_url: https://movie.douban.com/top250
    pages: 10
    page_size: 25
    delay_seconds: 5
    schedule: "0 6 * * 1"   # 每周一 06:00
```

代码 7-3 读取配置

```python
import yaml

with open("config/sources.yaml", encoding="utf-8") as f:
    sources = yaml.safe_load(f)["sources"]

douban = next(s for s in sources if s["name"] == "douban_top250")
print(douban["start_url"], douban["pages"])
```

7.4 调度：从 cron 到工作流

表 7-2 调度方式选择

方式	适合	注意
Windows 任务计划 / cron	单机、简单周期任务	注意时区、虚拟环境路径、日志
Prefect / Dagster	多步骤、依赖、重试	学习成本中等；适合研究团队
Airflow	大规模、多人维护	较重；小项目不必上
GitHub Actions	公开数据、轻量任务	运行时间/网络限制；注意密钥

调度不是“每天跑一次”这么简单。至少要有：失败重试、超时、并发限制、运行日志、失败告警，以及“本次运行抓到多少条”的记录。豆瓣榜单变化慢，每周一次足够。

7.5 存储：不要用 CSV 当数据库

表 7-3 数据分层与格式

层	格式	用途
原始层	HTML / JSON	证据与重放；按日期和哈希命名
中间层	Parquet	类型明确、压缩好、读取快
分析层	Parquet / DuckDB / Postgres	面板数据、查询、合并
元数据	JSON / YAML	来源、字段、许可、抓取时间

CSV 适合交换和快速查看，但不适合作为唯一存储：它没有类型、没有 schema、追加写入容易出错。用 Parquet 保存中间层，用 DuckDB 直接查询 Parquet，是单机研究项目很实用的组合。

代码 7-4 用 DuckDB 查询豆瓣榜单

```
import duckdb

con = duckdb.connect()
df = con.sql("""
    SELECT rank, title, rating, quote
    FROM read_parquet("data/interim/douban_top250.parquet")
    WHERE rating >= 9.0
    ORDER BY rank
""").df()
```

7.6 日志与监控

日志至少包含：运行 ID、数据源、URL、状态码、耗时、解析条数、错误类型。不要只打印 `done`。对“条数为 0”设置告警，因为它通常意味着页面结构变了或被替换成验证页。

代码 7-5 结构化日志的最小配置

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
    handlers=[
        logging.FileHandler("logs/run.log", encoding="utf-8"),
        logging.StreamHandler(),
    ],
)
log = logging.getLogger("douban_top250")
log.info("fetched rows=%s", len(rows))
```

7.7 测试：用固定 HTML 做回归

爬虫测试不应该依赖实时网络。把一份豆瓣榜单 HTML 保存到 `tests/fixtures/`，用解析函数读它，断言字段数量、类型和关键值。页面改版时，测试会先失败，而不是等到数据出错。

代码 7-6 用固定 HTML 做单元测试

```
from pathlib import Path
from bs4 import BeautifulSoup

def parse_titles(html: str):
    soup = BeautifulSoup(html, "lxml")
    return [
        c.select_one(".hd a .title").get_text(strip=True)
        for c in soup.select("ol.grid_view > li")
    ]

def test_parse_titles():
    html = Path("tests/fixtures/top250.html").read_text(encoding="utf-8")
    titles = parse_titles(html)
    assert len(titles) == 25
    assert titles[0] == "肖申克的救赎"
```

测试至少覆盖：正常页面、缺短评页面、空页面、编码异常页面。这样选择器改动时，你能立刻知道影响了什么。

7.8 部署：从笔记本到服务器

- **路径**：服务器上用绝对路径或项目根目录，不要依赖“当前目录”。
- **时区**：榜单按北京时间更新？cron 按哪个时区运行？统一写进 README。
- **网络**：服务器 IP 可能被豆瓣限制；优先低频，不要靠代理硬绕。
- **资源**：Playwright 需要浏览器和内存；给容器设置上限，避免拖垮整机。
- **日志**：挂载日志目录；容器重启后还能看到上次失败原因。

7.9 小结

工程化的目标不是“看起来专业”，而是让下一次运行更省力、让错误更早暴露、让数据可追溯。下一章把这些要求落到“数据质量与可复现性”上。

第 8 章

数据质量与可复现性

榜单会变、评分会变、页面会改。能复现的研究，必须留下“当时看到了什么”的证据。

本章目标

- 保存豆瓣榜单的原始快照、哈希和抓取时间
- 建立数据字典与校验规则
- 把一次抓取打包成可复现的材料

抓取到的数据不是“真相”，而是“某个时刻、某个页面、某个脚本看到的内容”。豆瓣 Top250 的排名、评分、评价人数每天都在变；如果不在当时保存证据，几个月后你很难解释：为什么这个排名和现在不一样？

8.1 原始快照：最小成本的可复现保险

每次请求都保存原始 HTML，按“日期/页码/哈希”命名。不要只保存清洗后的 CSV；原始 HTML 才是重放的依据。

代码 8-1　豆瓣快照的命名示例

```
data/raw/douban_top250/
├─ 2026-10-27_start0.html
├─ 2026-10-27_start0.manifest.json
├─ 2026-10-27_start25.html
└─ ...
```

代码 8-2 保存快照与 manifest

```
import datetime
import hashlib
import json

raw = resp.content
digest = hashlib.sha256(raw).hexdigest()
stamp = datetime.datetime.now(datetime.timezone.utc).strftime("%Y-%m-%dT%H-%M-%SZ")

with open(f"data/raw/douban/{stamp}.html", "wb") as f:
    f.write(raw)

manifest = {
    "url": resp.url,
    "fetched_at": stamp,
    "status": resp.status_code,
    "sha256": digest,
    "bytes": len(raw),
}
with open(f"data/raw/douban/{stamp}.manifest.json", "w", encoding="utf-8") as f:
    json.dump(manifest, f, ensure_ascii=False, indent=2)
```

manifest 里至少要有 URL、抓取时间、状态码、哈希、字节数。哈希用于确认文件没有被改动；时间用于还原“当时能看到什么”。

8.2 数据字典

数据字典不是可选项。它应该让另一个人不看代码也能理解每个变量的含义、单位、来源和缺失规则。

表 8-1 豆瓣 Top250 数据字典

字段	含义	类型/单位	来源	缺失规则
rank	榜单排名	整数 1—250	.pic em	不缺失
title	中文片名	字符串	.hd a .title	不缺失
rating	豆瓣评分	0—10 小数	.rating_num	不缺失
quote	一句话短评	字符串	.quote .inq	缺失记空字符串
url	详情页链接	URL	.hd a	不缺失
fetched_at	抓取时间	UTC ISO8601	脚本生成	不缺失

如果变量经过变换（比如把评分分段、把短评分词），把变换公式和参数也写进字典，不要只留一个“processed”文件。

8.3 校验规则

表 8-2 豆瓣榜单的校验规则

检查	例子	失败时
行数	本次 250 行，上次 250 行	告警；检查分页/选择器
唯一键	`rank` 1—250 不重复	去重或停止
类型	`rating` 可转 float	记录异常行
范围	`rating` 在 0—10；`rank` 在 1—250	人工复核
连续性	排名从 1 到 250 无缺口	检查翻页/解析
空值率	`title`、`rating` 无缺失	检查选择器/页面
链接	`url` 唯一且包含 `/subject/`	检查选择器

代码 8-3 用断言做最小校验

```
assert len(df) == 250, "条数不是 250"
assert df["rank"].is_unique, "排名重复"
assert df["rank"].between(1, 250).all(), "排名越界"
assert df["rating"].between(0, 10).all(), "评分越界"
```

断言适合“硬约束”；对于“评分均值比上次低 0.1”这类软约束，写入日志并告警，不要直接让任务失败。

8.4 榜单的“版本”：每次抓取都是一个 vintage

豆瓣 Top250 的排名、评分、评价人数会随时间变化。今天抓到的第 1 名，三个月后可能变成第 2 名；9.7 分也可能变成 9.6。研究中使用榜单时，必须记录抓取日期，把它当成一个“版本”（vintage），而不是永恒不变的真相。

- **排名变化**：名次会变；不要用今天的榜单解释一年前的现象。
- **评分变化**：评分和评价人数会变；比较时注明抓取日期。
- **页面变化**：字段、类名、分页方式都可能变；保存 HTML 才能回溯。

> **榜单数据的前视偏差**
>
> - 如果用“今天”的 Top250 去回测“过去”的票房或口碑，会引入前视偏差。
> - 解决方式是保存每次抓取的快照，并按抓取日期使用对应版本。

8.5 人工抽检

自动校验只能发现“形式错误”，发现不了“语义错误”：片名错位、评分对应错、把原名当中文名。全量抓完后，随机抽 30—50 条，回到豆瓣页面逐字段核对，记录错误率。

表 8-3 人工抽检清单

抽检项	怎么做	通过标准
字段完整性	每条记录的排名/片名/评分是否为空	关键字段 100% 非空
数值正确性	与页面逐条对比评分	错误率 < 1%
排名连续性	1—250 是否有缺口/重复	无缺口、无重复
链接有效性	随机打开 10 个 `url`	可访问或已存档

8.6 复现包

论文投稿或组会汇报时，至少打包以下内容。豆瓣数据可以公开，但也要注意不要重新分发完整榜单；代码、字段说明和抓取日期必须能公开。

- **代码**：取数、清洗、校验、作图脚本；按运行顺序编号。
- **环境**：`requirements.txt` 或 `environment.yml`；Python 版本。
- **原始快照**：可公开的 HTML 快照；不能公开的说明获取方式与限制。
- **数据字典**：字段、类型、来源、缺失规则、变换公式。
- **README**：一张流程图、运行命令、预期输出、常见错误。
- **引用**：豆瓣电影 Top250 的 URL、访问日期、抓取范围。

代码 8-4 一个最小复现包

```
douban_top250_replication/
├─ README.md
├─ requirements.txt
├─ config/sources.yaml
├─ src/01_fetch.py
├─ src/02_clean.py
├─ src/03_validate.py
├─ data/raw/            # 可公开的 HTML 快照
├─ data/processed/      # 分析数据
└─ docs/data_dictionary.md
```

8.7 小结

可复现不是“把代码传到网盘”，而是让另一个人能在不看你的情况下，理解数据从哪来、经过了什么、为什么可信。下一章讨论取数的边界：哪些事不能做，为什么不能做。

第 9 章

合规、伦理与风险

能抓到，不等于可以抓；能用来做研究，也不等于可以公开。技术方案必须放进法律与伦理的边界里。

本章目标

- 读懂 robots.txt 与服务条款的基本约束
- 识别版权、数据库权利和个人信息风险
- 为自己设定“什么时候停”的规则

这一章不是法律意见，而是研究者的风险清单。不同国家、不同机构、不同数据源的规则差异很大；在正式项目开始前，请让所在机构的法务、伦理审查或数据管理部门确认。

9.1 robots.txt：站点的抓取意愿

robots.txt 放在站点根目录，用 `User-agent`、`Disallow`、`Allow`、`Crawl-delay` 等字段告诉爬虫哪些路径可以访问。它是行业约定，不是法律本身；但无视它，可能构成对服务条款的违反，也会让整个研究机构被屏蔽。

代码 9-1　豆瓣 robots.txt 的节选

```
User-agent: *
Disallow: /subject_search
Disallow: /search
Disallow: /people/*/collect
Allow: /ads.txt
# Crawl-delay: 5
```

代码 9-2　用标准库检查豆瓣 Top250

```
from urllib.robotparser import RobotFileParser

rp = RobotFileParser()
rp.set_url("https://movie.douban.com/robots.txt")
rp.read()
print(rp.can_fetch("*", "https://movie.douban.com/top250"))
```

注意：robots.txt 允许抓取，不代表允许存储、再分发或商业使用；也不代表数据不受版权或个人信息规则约束。

豆瓣案例的具体边界

- `/top250` 不在 Disallow 列表里，但 `Crawl-delay: 5` 被注释掉了；本书仍按 5 秒执行。
- 只取榜单公开字段，不登录、不抓用户信息、不绕过 403/验证码；被拒绝就停止。
- 不要把完整榜单做成对外服务或商业产品；研究和教学使用时注明来源与抓取日期。

9.2 服务条款：合同层面的约束

注册、登录、使用 API 时，你通常已经同意了服务条款。条款可能限制：抓取频率、并发数、存储时长、二次分发、商业使用、账号共享。违反条款的后果包括封号、民事索赔，甚至机构层面的访问限制。

- **API 条款**：看清配额、署名、缓存、再分发、商业用途。
- **网站条款**：看清是否禁止自动化访问、是否允许保存副本。
- **机构订阅**：Wind、CEIC、CSMAR 等通常禁止把数据转给第三方；先问图书馆/数据库管理员。
- **账号**：不要共享账号、不要绕过并发限制；登录态文件等同凭证。

9.3 版权与数据库权利

事实本身通常不受版权保护，但事实的“选择、编排、表达”可能受保护；表格、报告、新闻、图表尤其如此。欧盟还有独立的数据库权利（sui generis），即使内容本身是事实，也可能禁止实质性提取。

表 9-1 常见版权与数据库风险

对象	风险点	相对安全的做法
统计表格	整表复制、再分发	引用来源；只取研究所需字段；遵守许可
新闻/报告	大段正文、翻译	引用短句并注明；用 API/授权；不替代原作品
图表/图片	复制、再发布	自己重绘；或取得授权
数据库	实质性提取、重复利用	购买许可；只做研究分析；不公开原库
代码/网页	复制整站结构	只取数据；不复制受保护的代码/设计

9.4 个人信息：公开不等于可随意使用

姓名、邮箱、电话、身份证号、住址、账号、简历、社交发言等属于个人信息；部分还可能构成敏感个人信息。《个人信息保护法》《数据安全法》以及 GDPR 等，对收集、存储、使用、共享都有要求。

- **最小必要**：只收集研究问题真正需要的字段，不抓“以后可能有用”的全量档案。
- **目的限定**：用于研究，不等于可以公开；发表时做去标识化/聚合。
- **安全**：原始数据加密/受限访问；不要放在公开仓库或共享网盘。
- **敏感信息**：健康、宗教、政治倾向、生物识别等，风险更高；没有明确依据不要采集。
- **公开可见**：公开网页上的信息仍可能受个人信息规则约束；“能看到”不是“随便用”。

9.5 平台边界：什么时候必须停

下面这些行为，本指南明确不建议，也不提供实现方法。它们不只是技术问题，而是可能带来法律、伦理和机构声誉风险。

- 绕过登录、付费墙、验证码、地区限制或平台风控。
- 使用窃取/共享的账号、Cookie、API key。
- 伪造身份、伪造 UA 来冒充普通用户，或隐藏自动化行为。
- 在被明确拒绝、收到警告或封禁后，换 IP/账号继续抓取。
- 抓取并公开个人简历、联系方式、社交关系等隐私数据。

> **一个简单的停止规则**
>
> - 如果站点要求登录、出现验证码、返回 403/451、或条款明确禁止自动化访问：停下来，找 API、申请授权、购买数据，或换研究设计。

9.6 研究者行为准则

合规不仅是“不违法”，也是一套可被同行审查的行为习惯。

- **标识身份**：User-Agent 写项目名和联系邮箱；必要时提前联系数据提供方。
- **限速**：串行或低并发；避开高峰；遵守 `Crawl-delay`。
- **缓存**：同一页面不重复请求；用本地快照重放。
- **不转售**：不把抓到的数据做成商业产品；不公开受许可限制的原始库。
- **可审计**：记录 URL、时间、脚本版本、许可依据；论文附录写清楚。
- **可撤回**：如果数据提供方要求删除，保留删除记录并更新衍生数据。

9.7 风险矩阵

表 9-2 取数场景的风险矩阵

场景	法律/条款风险	技术/声誉风险	建议
豆瓣 Top250，低频，公开字段	低	低	教学可用；5 秒间隔，保存快照
公开统计表，低频，注明来源	低	低	可以做；保存快照
公开新闻正文，批量存档	中	中	用 API/授权；只存必要片段
登录后下载机构订阅数据	中高	中	遵守订阅许可；不共享账号
绕过验证码/付费墙	高	高	不做
抓取个人简历/联系方式	高	高	不做；除非有明确伦理批准与法律依据
大规模抓取电商价格	中高	高	优先商业数据；评估条款与竞争法

9.8 合规检查清单

表 9-3 合规检查清单

检查项	通过标准
robots.txt	已读；不抓 Disallow 路径；遵守 Crawl-delay
服务条款	已确认允许自动化访问、存储和二次分析
版权/数据库	不复制受保护表达；不公开原库；引用来源
个人信息	最小必要；去标识化；安全存储；伦理审查
访问边界	不绕过登录/付费墙/验证码/风控；被拒即停
频率	低并发、有间隔、可缓存；不影响站点
身份	UA 有项目名与联系方式；必要时提前沟通
记录	URL、时间、许可、脚本版本可追溯

第 10 章

常见问题与排查手册

先复现、再定位、后修复。不要用“换代理/换 UA/加 sleep”掩盖真正的问题。

本章目标

- 按固定顺序定位 403、429、空结果、乱码和超时
- 区分“没权限”“没找到”“没渲染”三类问题
- 把一次排错变成可复用的测试样本

爬虫报错的信息往往很少：一个 403、一个空列表、一个 `NoneType`。有效的排查不是不断试参数，而是把问题缩小到“请求—响应—解析—保存”的某一环。

10.1 症状对照表

表 10-1 常见症状与第一反应

现象	最可能的原因	先做什么
401/403	缺 key、无权限、UA 被拒、地区限制	看文档/条款；确认授权；不要绕
429	请求太快或并发太高	读 `Retry-After`；降速/退避
200 但无数据	JS 渲染、选择器错、登录过期	看 View Source 与 Network
Timeout	网络慢、页面重、限流	加 timeout、重试、减并发
SSL 错误	公司代理/证书链	修证书；不要关闭验证
中文乱码	编码判断错误	显式设 `resp.encoding`
`NoneType` 报错	选择器没匹配到	保存 HTML；在浏览器验证
Scrapy 0 items	回调名错、allowed_domains、robots	看日志 Crawled/Scraped
Playwright 超时	选择器错、iframe、未登录	截图/trace；先等 body
下载失败	新窗口、接口触发、权限	监听 download/response
CSV 乱码	Excel 需要 BOM	`utf-8-sig`
重复行	翻页重复/详情页重复	唯一键去重
本地能跑、服务器不行	IP、字体、无头、时区	trace + 日志；不要伪造指纹

10.2 固定排查顺序

1. **只保留一个 URL**：把循环、并发、代理全部关掉，先让单条请求稳定。
2. **打印响应元信息**：状态码、最终 URL、编码、content-type、正文前 500 字。

3. **保存 HTML**：写到 `debug.html`，后续都在本地文件上测试，避免反复请求。
4. **在浏览器验证选择器**：用 Elements/Console 数节点；确认不是 JS 渲染。
5. **对比 View Source 与 Elements**：源代码没有、Elements 有，就是动态页面。
6. **看 Network**：是否有 XHR/Fetch 返回 JSON；能否用 requests 复现。
7. **检查权限与条款**：401/403/验证码/登录，先解决授权，不要技术硬绕。
8. **记录并停止**：把失败 URL、状态、时间写进日志；超过重试上限就停。

代码 10-1 最小复现脚本

```python
import requests

resp = requests.get(url, timeout=30)
print(resp.status_code, resp.url, resp.encoding)
print(resp.headers.get("content-type"))
print(resp.text[:500])

with open("debug.html", "wb") as f:
    f.write(resp.content)
```

10.3 三类问题，不要混为一谈

表 10-2 三类问题的区分

类型	特征	解决方向
没权限	401/403、登录页、验证码、451	API/授权/购买；停止
没找到	200、HTML 正常、选择器为空	改选择器；看 DOM/iframe
没渲染	View Source 无数据、Network 有 XHR	requests 取 JSON 或 Playwright

不要用“技术手段”解决“权限问题”

- 403 不是“需要换 UA”，429 不是“需要换代理”，验证码不是“需要识别”。
- 先确认你有没有被允许访问；没有授权时，正确动作是停止或换数据源。

10.4 Scrapy 与 Playwright 的专项排查

Scrapy 0 items：先看日志里 `Crawled` 是否为 0。如果为 0，检查 `start_urls`、`allowed_domains`、`ROBOTSTXT_OBEY`；如果 `Crawled > 0` 但 `Scraped = 0`，检查 `callback` 名称、选择器、Pipeline 是否 `DropItem`。

Playwright 超时：先截图，确认页面是否真的打开；再检查目标是否在 iframe/shadow DOM；最后确认登录态是否过期。不要用 `wait_for_timeout(10000)` 代替定位。

下载失败：确认下载是由点击、还是由新窗口/接口触发；用 `expect_download` 包住动作；检查浏览器是否允许自动下载。

10.5 把排错变成测试

每次修复一个页面结构问题，都把当时的 HTML 保存到 `tests/fixtures/`，并加一个断言。下次页面再变，测试会先失败，而不是论文数据悄悄少了一半。

代码 10-2　为豆瓣排错结果加一个回归测试

```
def test_top250_rows():
    html = Path("tests/fixtures/top250.html").read_text(encoding="utf-8")
    rows = parse_cards(html)
    assert len(rows) == 25
    assert rows[0]["title"] == "肖申克的救赎"
```

10.6 什么时候该放弃

- 需要绕过登录、付费墙、验证码或明确的风控。
- 站点条款禁止自动化访问，且没有授权渠道。
- 维护成本已经超过数据价值：每周都在改选择器。
- 存在更稳定的替代源：API、年鉴、商业数据库、合作方。

放弃不是失败。研究目标是“拿到可信数据”，不是“证明我能抓到”。

第 11 章

练习与进阶路线

看懂不等于会写。下面五个练习都围绕豆瓣 Top250，每个都有明确的验收标准。

本章目标

- 完成一次豆瓣请求、快照与元数据记录
- 用 BS4 和 Scrapy 各抓 250 条
- 用 Playwright 翻两页，并解释何时不需要浏览器

建议按顺序做。每个练习都要求保存原始 HTML、写 README、做一次校验；不要只交一个 CSV。

11.1 练习一：请求、快照与 manifest

任务：用 `requests` 抓取豆瓣 Top250 第一页，保存 HTML 与 manifest，打印状态码、字节数和条目数。

- URL：`https://movie.douban.com/top250?start=0&filter=`；间隔 5 秒。
- 保存 `data/raw/douban/` 下的 HTML 与 `.manifest.json`（URL、时间、状态码、sha256、字节数）。
- 用 BeautifulSoup 数出 `ol.grid_view > li` 的 25 个条目。

验收：README 写明访问日期；manifest 哈希可复算；控制台显示 25。

11.2 练习二：requests + BeautifulSoup

任务：抓取豆瓣 Top250 全部 10 页，输出 `rank, title, rating, quote, url`。

- 用 `start=0..225` 翻页；请求间隔 5 秒；串行。
- 保存原始 HTML 到 `data/raw/douban/`，CSV 到 `data/processed/`。
- 校验：250 条；排名 1—250 连续；评分 0—10；URL 唯一。

验收：重跑条数一致；日志显示 10 页与 250 条。

11.3 练习三：Scrapy

任务：用 Scrapy 抓取同一份豆瓣 Top250，输出 JSONL，并启用 `JOBDIR`。

- 项目名 `douban_top250`，Spider 名 `top250`；Pipeline 把排名/评分转成 int/float。

- 设置 `DOWNLOAD_DELAY=5`、`CONCURRENT_REQUESTS=1`、`ROBOTSTXT_OBEY=True`。
- 校验：250 条；排名连续；无重复 URL。

验收： `scrapy crawl top250 -O data/douban_top250.jsonl` 可复现；中断后能续。

11.4 练习四：Playwright

任务： 用 Playwright 打开豆瓣 Top250，抓取前两页，保存一张截图。

- 用 `wait_for_selector("ol.grid_view > li")`，不要用固定 `sleep`。
- 点击 `.paginator .next a`，等待第二页的排名变成 26。
- 对比 `requests.get()` 同一个 URL 的 HTML，说明豆瓣为什么其实不需要浏览器。

验收： 脚本无头可跑；截图能看到 25 个条目；README 写出两种方案的取舍。

11.5 练习五：数据字典与合规说明

任务： 为豆瓣 Top250 项目写数据字典、README 和合规说明，并打包一个最小复现包。

- 数据字典：`rank`、`title`、`rating`、`quote`、`url` 的含义、类型、来源、缺失规则。
- 合规说明：robots.txt 依据、5 秒间隔、只取公开字段、不登录、不绕过风控。
- 复现包：代码、`requirements.txt`、原始快照、处理后的 CSV、运行命令。

验收： 另一个同学能按 README 复现 250 条；合规说明能回答“为什么可以取”。

11.6 30 天学习计划

表 11-1 四周学习计划

周	重点	每天 1—2 小时做什么	周末产出
第 1 周	Python/HTTP/HTML	变量、循环、函数；URL/状态码；标签与 DOM	用 DevTools 找到 3 个选择器
第 2 周	requests + BS4	请求、编码、卡片、翻页、CSV	完成练习二
第 3 周	Scrapy	项目、Spider、Pipeline、settings、JOBDIR	完成练习三
第 4 周	Playwright + 工程	等待、点击、截图；目录与校验	完成练习四、五

11.7 进阶路线

- **异步与性能：** `httpx`、`asyncio`、Scrapy 并发调优；先保证正确，再谈速度。

- **文档解析**：PDF 表格（Camelot、pdfplumber）、版面分析（Docling、MinerU）、OCR。
- **文本数据**：正文抽取（trafilatura）、去重、主题模型、LLM 结构化抽取与人工校验。
- **数据工程**：DuckDB/Postgres、Parquet、Prefect/Dagster、数据质量测试。
- **研究设计**：把“抓取到的样本”与“目标总体”对齐，处理覆盖偏差和缺失。

11.8 最后三句话

先用 API，再写爬虫。 能买/能申请/能下载的数据，不要用抓取解决。

先学 HTML，再学框架。 选择器是核心，框架只是把循环工程化。

先保存证据，再谈分析。 原始快照、抓取时间、脚本版本，是研究可复现的底线。

附录 A

环境安装与命令速查

以下命令以 Windows PowerShell 为主；macOS/Linux 的差异已标注。

A.1 安装 Python 与虚拟环境

代码 A-1 Windows 下的虚拟环境

```
python --version
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
```

代码 A-2 macOS/Linux 下的虚拟环境

```
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

A.2 安装本书用到的库

代码 A-3 一次性安装

```
pip install requests beautifulsoup4 lxml pandas
pip install scrapy
pip install playwright
playwright install chromium
```

如果只需要其中一种方案，不必全部安装。Scrapy 和 Playwright 的体积较大，按需安装即可。

A.3 验证安装

代码 A-4 检查版本

```
import requests, bs4, scrapy, playwright, pandas

print("requests", requests.__version__)
print("bs4", bs4.__version__)
print("scrapy", scrapy.__version__)
print("pandas", pandas.__version__)
print("playwright ok")
```

A.4 VS Code 最小配置

- 安装 Python 扩展；用 `Ctrl+Shift+P` → “Python: Select Interpreter” 选择 `.venv`。
- 打开集成终端，确认提示符前有 `(.venv)`。
- 安装 Rainbow CSV（可选），查看 CSV 更直观。
- 把 `data/`、`.env`、`auth.json` 加入 `.gitignore`。

A.5 常见安装问题

表 A-1 安装阶段的常见问题

现象	原因	处理
`python` 不是内部或外部命令	PATH 未配置	重装并勾选 Add Python to PATH；或用 `py -3`
`pip` 安装超时	网络或镜像慢	换国内镜像；或加 `--default-timeout=100`
`playwright install` 失败	浏览器下载/代理	重试；Linux 加 `--with-deps`；公司网络配代理
`ImportError` / 找不到包	解释器选错、venv 未激活	VS Code 选 `.venv`；终端确认提示符有 `(.venv)`
`scrapy` 命令找不到	Scripts 不在 PATH	用 `python -m scrapy`，或先激活虚拟环境
权限错误	装到了系统目录	在 venv 中安装；不要用管理员权限

附录 B

选择器与代码片段速查

B.1 HTTP 状态码

表 B-1 状态码速查

状态码	含义	常见处理
200	成功	解析
301/302	重定向	跟随；检查最终 URL
400	参数错误	检查 URL/参数
401/403	未认证/禁止	检查 key/权限；不要绕
404	不存在	检查路径/资源
429	请求过多	退避；降速
5xx	服务器错误	有限重试

B.2 豆瓣 Top250：四个库的最小写法

代码 B-1 requests

```
# requests
resp = requests.get(
    "https://movie.douban.com/top250?start=0&filter=",
    headers=HEADERS, timeout=30,
)
resp.raise_for_status()
html = resp.text
```

代码 B-2 BeautifulSoup

```
# BeautifulSoup
soup = BeautifulSoup(html, "lxml")
cards = soup.select("ol.grid_view > li")
titles = [c.select_one(".hd a .title").get_text(strip=True) for c in cards]
```

代码 B-3 Scrapy

```
# Scrapy
for card in response.css("ol.grid_view > li"):
    yield {
        "rank": card.css(".pic em::text").get(default="").strip(),
        "title": card.css(".hd a .title::text").get(default="").strip(),
        "rating": card.css(".rating_num::text").get(default="").strip(),
    }
```

代码 B-4 Playwright

```
# Playwright
page.goto(url, wait_until="domcontentloaded")
page.wait_for_selector("ol.grid_view > li")
cards = page.locator("ol.grid_view > li").all()
```

B.3 豆瓣 Top250 选择器速查

表 B-2 豆瓣案例常用选择器

目标	选择器
全部电影条目	`ol.grid_view > li`
排名	`.pic em`
中文片名	`.hd a .title`
评分	`.rating_num`
短评	`.quote .inq`
详情链接	`.hd a::attr(href)`（Scrapy）
下一页	`.paginator .next a`

附录 C

术语表

表 C-1　常用术语

术语	解释
API	应用程序接口；按约定请求数据，通常返回 JSON
Endpoint	API 的具体地址
JSON	键值对结构的数据格式
HTML	网页的标记语言
DOM	浏览器解析 HTML 后形成的树
CSS 选择器	定位 DOM 节点的表达式
XPath	另一种节点定位语言，适合复杂层级
HTTP	浏览器/程序与服务器通信的协议
Status code	服务器返回的结果码
User-Agent	请求头中标识客户端的信息
robots.txt	站点根目录的抓取约定文件
Rate limit	单位时间内的请求上限
Pagination	分页；一次只返回一部分数据
Headless	不显示界面的浏览器模式
Spider	Scrapy 中定义抓取逻辑的类
Pipeline	Scrapy 中清洗/存储数据的组件
Snapshot	某一时刻保存的原始响应
Vintage	数据在某一时点的版本（含当时修订）

附录 D

经济学数据源入口

以下仅列出常见入口，具体许可、配额和字段以官方文档为准。

表 D-1 常见数据源

来源	入口	类型	备注
World Bank	api.worldbank.org/v2/	API + 批量	宏观、国别面板
FRED / ALFRED	fred.stlouisfed.org/docs/api/	API	美国与跨国序列；支持 vintage
IMF	data.imf.org	API/SDMX	国际收支、财政、金融
OECD	data.oecd.org	API/SDMX + 批量	发达国家面板
Eurostat	ec.europa.eu/eurostat	API + 批量	欧洲统计
BIS	data.bis.org	API/SDMX	银行、跨境、汇率
UN Comtrade	comtradeplus.un.org	API	双边贸易；注意配额
DBnomics	db.nomics.world	聚合 API	发现数据；正式引用回原源
SEC EDGAR	sec.gov/edgar	API + 全文	美国上市公司披露
GDELT	gdeltproject.org	批量/API	全球新闻事件
豆瓣 Top250（教学）	movie.douban.com/top250	网页/HTML	本书贯穿案例；非研究数据源
国家统计局	data.stats.gov.cn	网页/导出	中国宏观；优先导出
中国人民银行	pbc.gov.cn	网页/报告	金融、政策
巨潮资讯	cninfo.com.cn	网页/公告	A 股公告与 PDF
CSMAR / Wind	机构订阅	商业数据库	清洗好的面板；遵守许可

访问前先看“开发者/API/数据服务”页面和 terms；很多机构提供免费 key，但要求署名或限制频率。本表不构成授权，具体以数据提供方的最新条款为准。

附录 E

检查清单

E.1 抓取前

表 E-1 抓取前检查

	检查项
☐	研究问题需要哪些字段？最小必要集是什么？
☐	有没有 API、批量下载、机构订阅或公开数据附录？
☐	robots.txt 和服务条款是否允许？
☐	是否涉及个人信息、版权或数据库权利？
☐	是否准备了 UA、超时、重试、限速？
☐	是否先抓 10—50 条做样本？

E.2 抓取中

表 E-2 抓取中检查

	检查项
☐	状态码是否正常？429/5xx 是否退避？
☐	是否保存原始响应、时间、URL？
☐	翻页是否有停止条件、是否重复？
☐	日志是否能看出请求数、条数、错误数？
☐	是否遵守并发/频率限制？
☐	遇到 403/验证码/登录是否停止？

E.3 抓取后

表 E-3 抓取后检查

	检查项
☐	行数、唯一键、类型、范围是否通过？
☐	关键字段缺失率是否可接受？
☐	单位、频率、季节调整是否记录？
☐	是否随机抽检 30 条与原页面核对？

	检查项
☐	快照哈希、脚本版本、README 是否齐全？
☐	数据字典与引用信息是否完整？

E.4 代码审查

表 E-4 代码审查清单

	检查项
☐	代码/配置中没有明文 key、cookie、密码
☐	没有无限循环；重试有上限
☐	选择器有默认值；空结果会告警
☐	有固定 HTML 样本的回归测试
☐	数据写入是幂等的/可恢复的
☐	日志不打印敏感信息

把这份指南放在手边，但不要把它当成操作手册照抄。每个数据源都有自己的规则；先判断，再动手。